



מכללת כנפי רוח, קרית נוער ירושלים

סמל מוסד 140129

ספר פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון 714918)

בנושא:

פלטפורמה להעברת נתונים מקצה
לקצה בצורה מאובטחת SmallTalk



מגיש: מיכאל אוליאל

ת"ז: 214407017

מנחה: אילן פרץ

תשפ"ה 2025

תוכן עניינים

1	הצעת הפרויקט שאושרה ע"י משרד החינוך	3
2	מבוא	4
2.2	תהליך המחקר	4
2.3	סקירת ספרות ומאמרים	4
2.4	אתגרים מרכזיים במחקר	5
2.4.1	התמודדות עם הבעיה	5
2.4.2	הסיבות לבחירת הנושא	5
2.4.3	על איזה צורך הפרויקט עונה ?	5
2.4.4	הצגת הפתרונות שנבחרו לבעיה	5
3	מטרות ויעדים לפיתוח המערכת	5
	מטרות עיקריות:	5
	יעדים מרכזיים:	6
4	אתגרים בבניית המערכת	7
5	הגדרת מדדי הצלחה למערכת	9
6	רקע תיאורטי וספרות מקצועית	10
	קריפטוגרפיה:	10
7	תיאור מצב קיים וניתוח חלופות למערכת	13
	אפשרויות קיימות:	13
8	ויתור החלופה הנבחרת ונימוקים לבחירתה	15
10	תיאור התקיפה/ההגנה	20
10.2	תיאור הצפנות	22
11.1	ארכיטקטורת המערכת (ע"פ מודל 3 השכבות)	27
11.2	תיכון מפורט של רכיבי המערכת	28
11.3	מודל שרת/לקוח	29
11.4	תיאור מסד הנתונים	31
11.5	תהליכים במערכת ההפעלה	34
11.6	תיאור פרוטוקולי תקשורת	34
12	ניתוח תרחישים וזרימת המידע	36
12.2	תרשימי רצף SEQUENCE DIAGRAM	38
13	תיאור מסכים וממשק משתמש	45
13.1	תרשים היררכיית המסכים	45
13.2	תיאור המסכים	46
14	תיאור התוכנה	49
14.1	סביבת עבודה	49
14.2	שפות תכנות	49
14.3	תיאור המודולים והמחלקות	50
14.3.1	עץ המודולים	50
14.3.2	תרשים מחלקות Class Diagram	51
14.3.3	תיאור מחלקות מפורט	52
14.4	מבנה נתונים בשימוש	59
14.5	קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים	60
15	מדריך למשתמש	66
16	בדיקות והערכה	66
17	מסקנות	67
18	פיתוחים עתידיים	68
19	בבליוגרפיה	68

1 הצעת הפרויקט שאושרה ע"י משרד החינוך

פרטי המגיש

סמל מוסד: 140129

שם המכללה: כנפי רוח, קריית נוער ירושלים.

שם הסטודנט: מיכאל אוליאל ת"ז: 214407017

שם הפרויקט: SmallTalk פלטפורמה להעברת נתונים מקצה לקצה בצורה מאובטחת

פרטי הפרויקט

תיאור הפרויקט

תיאור הפרויקט SmallTalk היא אפליקציית Web לתקשורת מאובטחת. הפלטפורמה מספקת מענה לצורך הגובר בפרטיות ואבטחת מידע, באמצעות מנגנון הצפנת End-to-End האפליקציה מאפשרת שיחות פרטיות וקבוצתיות, שיתוף קבצים מוצפן בזמן אמת, ומבטיחה הגנה מלאה על הנתונים של המשתמשים ופרויקט זה שייך לתחום סייבר ואבטחת מידע וקריפטוגרפיה.

הגדרת הבעיה האלגוריתמית

ליבת הפרויקט מתמקדת בתחום **אבטחת סייבר וקריפטוגרפיה**, ובפרויקט זה אבנה יצירת מנגנון **הצפנה ופענוח** תוך שימוש באלגוריתמים מתקדמים כמו **ECDH** להחלפת מפתחות ו **AES-256** להצפנת התוכן המיועד להבטיח את פרטיות המשתמשים ולשליחת הודעות מוצפנות.

מבוא

בפרק זה מוצג הרקע לפרויקט SmallTalk, כולל הצורך בתקשורת דיגיטלית מאובטחת והפתרונות הקריפטוגרפיים שנבחרו למימושו. מתואר תהליך המחקר שבוצע, סקירת הספרות הרלוונטית והאתגרים שנפתרו במהלך הפיתוח. בנוסף, מפורטות הסיבות לבחירת הנושא והצגת הפתרונות: ECDH, AES-256 וחתימות דיגיטליות.

2.1 הרקע לפרויקט

הפרויקט SmallTalk נועד לתת מענה לצורך הגובר בתקשורת דיגיטלית מאובטחת בעידן שבו פרטיות המשתמשים נמצאת תחת איום מתמיד מצד גורמים זדוניים. הפלטפורמה משתמשת במנגנון הצפנה מקצה-לקצה (**End-to-End**) כדי להבטיח סודיות מוחלטת של הנתונים, ומציעה שירותים כמו שיחות פרטיות וקבוצתיות ושיתוף קבצים מוצפן בזמן אמת.

2.2 תהליך המחקר

תהליך המחקר כלל מספר שלבים:

1. **סקירת ספרות מקצועית** בתחום הקריפטוגרפיה ואבטחת מידע, תוך התמקדות באלגוריתמים כמו ECDH ו-AES-256.
2. **ניתוח צרכים** של משתמשים באפליקציות תקשורת מאובטחות.
3. **בחינת חלופות טכנולוגיות**, כולל RSA ו-DH-לבחירת הפתרון האופטימלי.
4. **פיתוח ובדיקת פתרונות** באמצעות שימוש במנגנונים מתקדמים להבטחת פרטיות וביצועים גבוהים.

2.3 סקירת ספרות ומאמרים

הפרויקט מתבסס על מחקרים שמוכיחים את יעילות השילוב בין ECDH להחלפת מפתחות לבין AES-256 להצפנה סימטרית:

- מחקר 1 הראה ששילוב זה משיג רמת אבטחה גבוהה תוך שמירה על יעילות ביצועית בסביבות מוגבלות משאבים כמו IoT.
- מחקר 2 הדגיש את היתרונות של מנגנונים קלים ויעילים שמאפשרים חיסכון בזמן ובמשאבים תוך שמירה על רמת אבטחה גבוהה.

2.4 אתגרים מרכזיים במחקר

2.4.1 התמודדות עם הבעיה

האתגרים המרכזיים היו:

- מניעת גישה לתוכן ההודעות מכל גורם חיצוני, כולל השרת עצמו.
- יצירת ערוץ תקשורת מאובטח להחלפת מפתחות בצורה דינמית ומוגנת.
- שילוב מנגנוני אבטחה מתקדמים עם מהירות תגובה גבוהה בזמן אמת.

2.4.2 הסיבות לבחירת הנושא

הנושא נבחר בשל הצורך הגובר בתקשורת דיגיטלית מאובטחת בעידן שבו פרטיות המידע נמצאת בסיכון מתמיד, לצד חשיבותו הרבה בתחום הסייבר והקריפטוגרפיה.

2.4.3 על איזה צורך הפרויקט עונה ?

הפרויקט עונה על הצורך בתקשורת מאובטחת המבטיחה סודיות מוחלטת של נתונים, תוך מתן אפשרות לשיתוף קבצים ושיחות קבוצתיות בצורה מוגנת ומוצפנת.

2.4.4 הצגת הפתרונות שנבחרו לבעיה

הפתרונות שנבחרו כוללים:

- **ECDH (Elliptic Curve Diffie-Hellman)** להחלפת מפתחות מאובטחת מעל ערוץ פתוח.
- **AES-256** להצפנה סימטרית ברמת אבטחה גבוהה במיוחד של תוכן ההודעות.
- **חתימות דיגיטליות**: לאימות זהות המשתמשים ולהבטחת שלמות ההודעות.

3 מטרות ויעדים לפיתוח המערכת

מטרות עיקריות:

1. **סודיות ההודעות**: מניעת גישה לתוכן ההודעות מכל גורם חיצוני, כולל השרת עצמו.
 2. **החלפת מפתחות מאובטחת**: יצירת ערוץ תקשורת שבו המפתחות המשותפים נוצרים בצורה דינמית ומוגנת.
 3. **אימות זהות**: מנגנון המאמת את זהות השולח והמקבל, ללא תלות בצד שלישי.
- ביצועים וסנכרון בזמן אמת**: שילוב אבטחה מתקדמת עם מהירות תגובה גבוהה.

יעדים מרכזיים:**א. ניהול משתמשים ואימות זהות:**

1. הרשמה מאובטחת ויצירת מפתחות קריפטוגרפיים אישיים (פרטי וציבורי).
2. אימות דו-שלבי (FA2) ושמירה על אבטחת חשבון המשתמש.

ב. יצירת ערוץ תקשורת מאובטח (בעיה אלגוריתמית)**1. ECDH להחלפת מפתחות:**

- כל משתמש יוצר זוג מפתחות (ציבורי ופרטי).
- מחושב מפתח משותף בטוח (Shared Secret) באמצעות המפתח הציבורי של הצד השני והמפתח הפרטי של המשתמש.

2. AES - 256 להצפנת הודעות:

- המפתח המשותף משמש להצפנה ופענוח של כל הודעה.
- הודעה מוצפנת נשלחת יחד עם IV (Initialization Vector) ייחודי.

ג. שליחת הודעות מוצפנות

- הודעות עוברות הצפנה סימטרית באמצעות AES-256.

ד. קבלת הודעות ופענוח

- המשתמש מקבל הודעה ומפענח אותה באמצעות המפתח המשותף.

ה. סנכרון בזמן אמת וניהול הודעות

- התראות והודעות מתעדכנות בזמן אמת באמצעות WebSockets.
- מנגנון לניהול היסטוריית הודעות ושמירת פרטיות המשתמש.

ו. מערכת גיבוי ושחזור מתקדמת

מערכת SmallTalk כוללת מערכת גיבוי ושחזור מתקדמת המהווה חלק מארכיטקטורת האבטחה הכוללת מערכת זו מבטיחה את שלמות הנתונים ואת המשכיות השירות תוך שמירה על עקרונות ההצפנה מקצה לקצה. רכיבי המערכת העיקריים:

1. מנגנון גיבוי אוטומטי מוצפן מערכת הגיבוי האוטומטית מבצעת גיבויים תקופתיים של כל המידע הקריטי במערכת, כולל היסטוריית שיחות, קבצים משותפים והגדרות משתמש. תהליך הגיבוי משתמש באותם עקרונות הצפנה של המערכת הראשית:
 - הצפנת נתונים באמצעות AES-256 עם מפתחות ייחודיים.
 - שימוש ב ECDH להחלפת מפתחות בתהליך השחזור.
 - חתימות דיגיטליות לאימות שלמות הגיבויים.

2. מערכת שחזור מאובטחת המערכת מספקת מנגנון שחזור מאובטח המאפשר למשתמשים לשחזר את המידע שלהם במקרה של אובדן מכשיר או תקלה. תהליך השחזור כולל:

- אימות זהות מחמיר באמצעות מנגנון דו-שלבי.
- פענוח הדרגתי של הנתונים המגובים.
- בדיקות שלמות ואימות של המידע המשוחזר.

3. מדיניות ניהול נתונים המערכת כוללת מנגנון חכם לניהול מדיניות שמירה והשמדה של נתונים:

- הגדרת תקופות שמירה לסוגי מידע שונים.
- מחיקה מאובטחת של נתונים שפג תוקפם.
- התראות אוטומטיות על מצב הגיבויים.

4 אתגרים בבניית המערכת

במהלך תכנון ובניית מערכת – SmallTalk – זוהו מספר אתגרים מרכזיים, טכנולוגיים ומהותיים, המחייבים פתרונות מתקדמים ומדויקת של רכיבי אבטחה, ביצועים וניהול. להלן האתגרים המרכזיים:

1. מידע מקצה לקצה (End-to-End Encryption)

המטרה המרכזית של המערכת היא להבטיח סודיות מלאה של המידע. האתגר נובע מהצורך לשלב מנגנוני הצפנה חזקים (AES-256) עם פרוטוקולי החלפת מפתחות – (ECDH) מבלי לאחסן את המפתחות על השרת. בנוסף, יש צורך למנוע גישה לא מורשית מצד גורמי ביניים כולל השרת עצמו, דבר המחייב הצפנה בצד הלקוח ופענוח רק אצל המקבל.

2. ניהול מאובטח של מפתחות קריפטוגרפיים

אחד האתגרים הקריטיים הוא כיצד לנהל ולשמור על מפתחות ההצפנה בצורה בטוחה. אם מפתח פרטי נחשף – כל ההודעות המוצפנות הופכות לפגיעות. לכן, יש צורך ביישום מנגנונים לשמירת מפתחות בזיכרון מאובטח, (secure storage) או לחלופין, חישוב דינמי של המפתח באמצעות ECDH בכל פתיחת שיחה.

3. תפקוד בזמן אמת וביצועים תחת עומס

מערכת התקשורת נדרשת לפעול בצורה מיידית וללא השהיות משמעותיות, גם בתנאים של ריבוי משתמשים. השימוש ב-WebSocket מאפשר תקשורת דו-כיוונית בזמן אמת, אך מצריך טיפול יעיל

בניהול חיבורים פתוחים רבים במקביל, מניעת עומס יתר על השרת, ואופטימיזציה של תהליכי הצפנה/פענוח שלא יפגעו בזמני תגובה.

4. סנכרון הודעות והיסטוריה מוצפנת

שמירה על היסטוריית שיחות בצורה מוצפנת מציבה אתגר כפול: גם לשמור על פרטיות התוכן, וגם לאפשר למשתמש לשחזר הודעות באופן מאובטח. בנוסף, אם משתמש עובר בין מכשירים – המידע המוצפן צריך להיות משוחזר תוך שמירה על סודיות מלאה, כולל מנגנוני גיבוי מוצפן ובדיקת שלמות החומר.

5. שילוב בין ממשקים וטכנולוגיות מגוונות

המערכת מבוססת על ארכיטקטורת שלוש שכבות Vaadin, בצד הלקוח Spring Boot, בצד השרת, ו-MongoDB-לאחסון. השילוב בין טכנולוגיות אלו מחייב תיאום מדויק בין הממשק, הלוגיקה והנתונים – במיוחד בנושאים של הצפנה ופענוח בשני הקצוות. חוסר תיאום עשוי לגרום לשגיאות, תקלות באימות החתימות הדיגיטליות, או חוסר תאימות בין גרסאות.

7. שמירה על פרטיות גם במידע מטא-דאטה

מעבר להצפנה של תוכן ההודעות, ישנו אתגר נוסף: הגנה על מידע עקיף – כמו תאריך השליחה, זהות השולח/מקבל, ומידע על הקבצים. אתגר זה מצריך הצפנה של המטא-דאטה או אחסונו במבנה שאינו מאפשר קישור ישיר למידע רגיש.

5 הגדרת מדדי הצלחה למערכת

כדי להעריך את הצלחת מערכת **SmallTalk** נדרשת הגדרה ברורה של מדדים (KPIs – Key Performance Indicators) המתייחסים לאיכות הפיתוח, האבטחה, הביצועים וחווית המשתמש. מדדים אלו נבחנים הן ברמה הטכנית והן ברמה הפונקציונלית.

אבטחה ופרטיות

- **100% הצפנה מקצה לקצה – (E2EE)** כל הודעה, קובץ ונתון מועברים מוצפנים, ונפענחים רק בצד הלקוח.
- **עמידה במבחני חדירה בסיסיים – (Penetration Tests)** המערכת תעבור בדיקות חדירה פנימיות להבטחת עמידות בפני תקיפות נפוצות (כמו, Man-in-the-middle, XSS, SQL Injection).
- **אימות מוצלח של חתימות דיגיטליות ב-100% מהמקרים** – כל הודעה חתומה תאומת על ידי הצד המקבל ללא שגיאות.

ביצועים וזמן תגובה

- **זמן שליחת הודעה ממוצע נמוך מ-100 ms** – באמצעות WebSocket והצפנה יעילה.
- **קיבולת של לפחות 100 משתמשים בו-זמנית ללא פגיעה בביצועים** – מדד זה נבחן באמצעות סימולציות עומס.
- **זמן הצפנה/פענוח ממוצע לכל הודעה – 50ms <** כולל תהליך יצירת מפתח, הצפנה, שליחה ופענוח.

חווית משתמש

- **ממשק משתמש תקין ונגיש** – לא נרשמות שגיאות בטעינה, כל הכפתורים פעילים ונגישים.
- **שיעור הצלחה של פעולות בסיסיות (התחברות, שליחת הודעה, שיתוף קובץ) $\geq 98\%$** .
- **עיצוב ידידותי ואינטואיטיבי** – לפי משוב ממשתמשים בניסוי קבלה

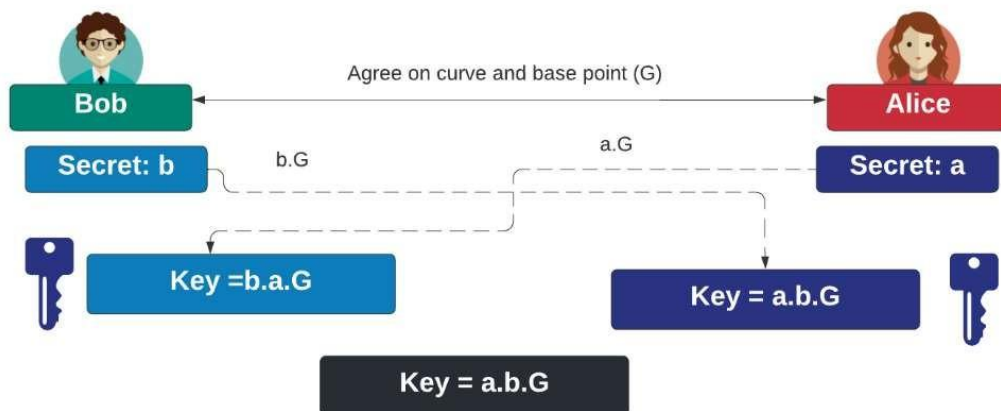
6 רקע תיאורטי וספרות מקצועית

קריפטוגרפיה:

בסעיף זה אסביר על אלגוריתמים ECDH ו AES 256 וחתימה דיגיטלית

• ECDH (Elliptic Curve Diffie-Hellman) אלגוריתם להחלפת מפתחות מאובטחת המבוסס על עקומות אליפטיות.

הפרוטוקול מאפשר לשני משתתפים שלא נפגשו מעולם ואינם חולקים ביניהם סוד משותף כלשהו מראש, להעביר אחד לשני מעל גבי ערוץ פתוח (שאינו מאובטח) סוד כלשהו כך שאיש מלבדם אינו יודע מהו. הסוד יכול להיות, בהקשר של פרוטוקול תקשורת מאובטח, מפתח הצפנה. פרוטוקול דיפי-הלמן מתמודד עם בעיית הפצת המפתחות על ידי פונקציה חד-כיוונית.



ניתן לראות באיור הנ"ל: ECDH הוא פרוטוקול לחילופין של מפתחות בין שני משתמשים,

המאפשר להם ליצור מפתח סודי משותף. הפרוטוקול עובד בשלושה שלבים עיקריים:

1. בוב ואליס מסכימים על עקום אליפטי ונקודת בסיס משותפים.

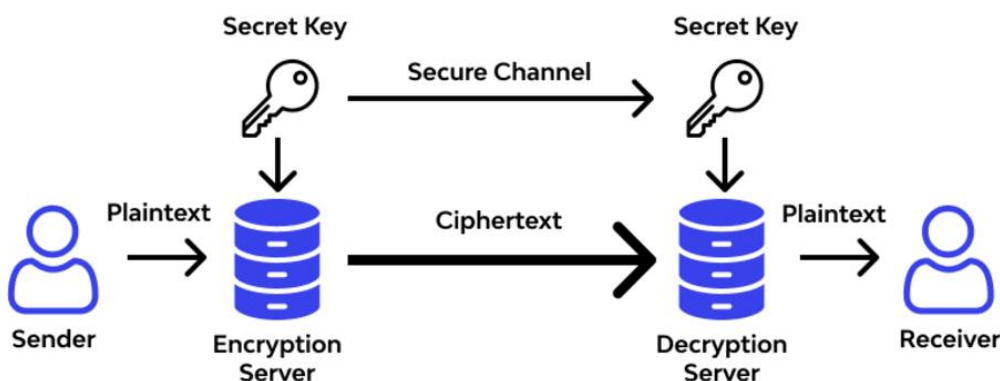
2. כל אחד מהם בוחר מספר סודי משלו, ומחשב מפתח ציבורי על בסיס זה.

3. הם חולקים את המפתחות הציבוריים, ואז כל אחד יכול לחשב את המפתח הסודי המשותף

באמצעות המפתח הציבורי של הצד השני והמפתח הסודי שלו התהליך מאפשר לבוב ואליס לשתף מפתח סודי מבלי שתוקף פוטנציאלי יוכל לחשב אותו, בזכות העובדה שהבעיה המתמטית שבבסיס זה היא מאוד קשה לפתרון.

• **AES-256 (Advanced Encryption Standard)** אלגוריתם סימטרי להצפנת הודעות, המספק רמת אבטחה גבוהה במיוחד הצפנת AES-256 היא דרך לשמור על הודעות סודיות או מידע בטוח מאנשים שלא אמורים להיות מסוגלים לראות אותם. הצפנת AES-256 היא כמו שיש לך מנעול סופר חזק על הקופסה שלך שניתן לפתוח רק על ידי מפתח מאוד ספציפי. המנעול כל כך חזק שיהיה קשה מאוד למישהו לשבור אותו ולפתוח את הקופסה ללא המפתח הנכון.

AES Algorithm Working



ניתן לראות באיור את התהליך של האלגוריתם ואיך הוא עובד :

AES הוא אלגוריתם הצפנה סימטרי, שמשמעותו שהשולח והמקבל משתמשים באותו מפתח סודי

1. השולח מזין את הטקסט הפשוט שהוא רוצה להעביר לשרת ההצפנה.
2. השרת משתמש במפתח הסודי כדי להצפין את הטקסט, ויוצר טקסט מוצפן.
3. הטקסט המוצפן נשלח דרך ערוץ מאובטח למקבל.
4. המקבל משתמש באותו מפתח סודי כדי לפענח את הטקסט המוצפן בחזרה לטקסט המקורי.

העיקרון הוא שהשולח והמקבל משתפים רק את המפתח הסודי ביניהם, מה שמאפשר להם להעביר מידע בצורה מאובטחת AES .

• **חתימות דיגיטליות**: מנגנון המבטיח את שלמות ההודעות ואותנטיות השולח

חתימה דיגיטלית היא שיטה **קריפטוגרפית**, שמטרתה לאמת את המקוריות והשלמות של הודעה או מסמך דיגיטלי.

חתימה דיגיטלית תקפה מעידה בסבירות גבוהה שההודעה או המסמך אינם משוברים או מזויפים, כלומר – שהמסמך או ההודעה נוצרו או נשלחו על ידי גורם

ידוע ושהם לא שונו על ידי צד שלישי לאחר חתימתם, או שובשו בדרך אחרת. כמו כן חותם המסמך לא יוכל להתכחש לכך שהוא יצר אותו.

ספרות מקצועית

- במאמר [1] יש מחקר המתאר יישום של ECDH ו AES בסביבות מוגבלות משאבים כמו . IoT . המחקר מראה ששילוב של ECDH ליצירת מפתחות ואימות עם AES להצפנה סימטרית משיג רמת אבטחה גבוהה תוך שמירה על יעילות ביצועית. בניסויים שיעור ההצלחה עמד על 99.9% וזמני ההצפנה הממוצעים היו מהירים ועמדו על 85 ms בניסיון הראשון ו 65 ms בניסיון השני. המסקנות מהמחקר תומכות בשילוב דומה בפרויקט SmallTalk כדי לספק הצפנה חזקה וביצועים מעולים.
- במאמר [2] המציג שילוב של ECDH להחלפת מפתחות ואימות עם AES להצפנה. המחקר מותאם למגבלות המשאבים של מערכות IoT ומדגיש את היתרונות של שימוש במנגנונים קלים ויעילים המאפשרים חיסכון בזמן ובמשאבים, תוך שמירה על רמת אבטחה גבוהה. ממצאי המחקר מחזקים את הבחירה בשיטות אלו בפרויקט SmallTalk כדי להתמודד עם אתגרי הביצועים והאבטחה.

7 תיאור מצב קיים וניתוח חלופות למערכת

אפשרויות קיימות:

1. Signal

- מבוססת על פרוטוקול Signal המשלב ECDH ו-AES
- דורשת מספר טלפון לזיהוי משתמשים, מוגבלת במנגנוני ניהול חשבונות.

2. WhatsApp

- משתמשת ב-Signal Protocol אך כוללת איסוף נתונים לצרכים מסחריים.

3. Telegram

- תומכת בהצפנה מקצה לקצה רק בצ'אטים פרטיים, עם דגש פחות על פרטיות מוחלטת.

בחנתי מספר גישות חלופיות להחלפת מפתחות ולהצפנת נתונים על מנת להבטיח את רמת האבטחה הגבוהה ביותר לצד ביצועים מיטביים. אלו האפשרויות שנבחנו:

- א. אלגוריתמים להחלפת מפתחות
 - RSA (Rivest-Shamir-Adleman)

שיטה מבוססת קריפטוגרפיה אסימטרית, המתבססת על הקושי שבפירוק לגורמים ראשוניים. RSA הוא פרוטוקול בוגר ומוכר, אך הוא פחות יעיל מבחינת ביצועים, בעיקר עם הגדלת אורך המפתח לצורך הבטחת רמת אבטחה מודרנית.
 - DH (Diffie-Hellman)

הגרסה המסורתית של פרוטוקול דיפי-הלמן. אמנם מספק אבטחה נאותה, אך אינו יעיל כמו ECDH, במיוחד בסביבות מוגבלות משאבים או מערכות עם דרישות ביצועים גבוהות.
 - ב. אלגוריתמים להצפנה סימטרית
 - ChaCha20

אלגוריתם הצפנה קל ומהיר יותר מ-AES. מתאים לסביבות עם מגבלות משאבים, אך פחות נפוץ בשימוש במערכות מבוססות תקנים תעשייתיים
 - DES (Data Encryption Standard)

אלגוריתם הצפנה סימטרי ותיק המשתמש באורך מפתח של 56 ביטים וצופן בלוקים בגודל 64 ביטים. DES היה תקן אבטחה פופולרי בשנות ה-70 וה-80, אך כיום נחשב למיושן ובלתי בטוח בשל אורך המפתח הקצר, שמאפשר לבצע התקפות כוח גס (force brute) בזמן סביר. בעוד שהוא פשוט וקל ליישום, הוא לא עומד בדרישות

האבטחה המודרניות.

• Triple DES

גרסה מורחבת של DES, המספקת רמת אבטחה גבוהה יותר. עם זאת, נחשב לפחות יעיל ופחות בטוח לעומת AES256, במיוחד אל מול התקפות מודרניות.

• ג. חתימות דיגיטליות

• RSA Digital Signature

שיטה מבוססת RSA ליצירת חתימות דיגיטליות. מספקת אבטחה טובה, אך פחות יעילה ויותר מסורבלת לעומת חתימות מבוססות ECDSA, המותאמות לקריפטוגרפיה על עקומות אליפטיות.

להלן השוואה בין החלופות על פי מאפיינים והתאמה לפרויקט זה

פרוטוקול/אלגוריתם	רמת אבטחה	ביצועים	התאמה לסביבות מוגבלות משאבים	התאמה לפרויקט SmallTalk
ECDH	גבוהה	גבוהה	טובה מאוד	מתאים מאוד
RSA	גבוהה	נמוכה	מוגבלת	מתאים פחות
DH (מסורתי)	בינונית	נמוכה	מוגבלת	מתאים פחות
AES-256	גבוהה	גבוהה	טובה	מתאים מאוד
ChaCha20	גבוהה	גבוהה מאוד	מצוינת	אופציונלי
Triple DES	בינונית	נמוכה	נמוכה	מתאים פחות

8 ותיאור החלופה הנבחרת ונימוקים לבחירתה

האלגוריתמים שנבחרו למימוש בפרויקט זה הבחירה באלגוריתמים ECDH ו-AES-256 נובעת מהשילוב המיטבי שהם מספקים בין אבטחה ביצועים ויכולת התאמה לצרכי הפרויקט:

ECDH

נבחר בזכות יעילותו הגבוהה בסביבות מודרניות, תוך שמירה על רמת אבטחה גבוהה. פרוטוקול זה מתאים במיוחד לסביבות מוגבלות משאבים כמו מערכות מבזרות ואפליקציות בזמן אמת.

AES-256

השימוש ב-AES-256 להצפנה סימטרית נבחר בשל השילוב האידיאלי בין פשטות יישום, תמיכה רחבה בתעשייה, ורמת אבטחה שאינה ניתנת לשבירה באמצעים ידועים.

בעוד RSA ו DH מהווים פרוטוקולים בוגרים ומוכרים, הם פחות יעילים מ ECDH בסביבות הדורשות ביצועים גבוהים. באותו אופן AES-256 נמצא מתאים במיוחד בזכות פשטות יישומו והאמינות הגבוהה שהוא מספק לאורך שנים

9 אפיון המערכת המוצעת

בפרק זה יוצג אפיון המערכת SmallTalk ברמה מערכתית, תוך פירוט הדרישות המרכזיות מהמוצר. הפרק יתאר את מבנה המודולים, היכולות הפונקציונליות והביצועים הנדרשים, לצד אילוצים טכנולוגיים ואבטחתיים. מטרת הפרק היא להגדיר את תשתית המערכת לפני שלב המימוש בפועל.

9.1 ניתוח הדרישות מהמערכת

המערכת SmallTalk נועדה לאפשר העברת מידע באופן מאובטח מקצה-לקצה בין משתמשים. הדרישות המרכזיות מהמערכת הן:

- תמיכה בריבוי משתמשים: המערכת צריכה לאפשר שימוש של מספר רב של משתמשים בו-זמנית, מבלי לפגוע בביצועים או באמינות.
- שמירה ואחזור של נתונים: נדרש מנגנון המאפשר שמירת מידע זמני או קבוע, תוך הקפדה על פרטיות וסודיות הנתונים.
- מהירות תגובה: על המערכת לספק תקשורת מהירה בין הצדדים, עם שיהוי מינימלי בזמן שליחת וקבלת מידע.
- זמינות ואמינות גבוהה: נדרש שהמערכת תפעל בצורה יציבה לאורך זמן ותהיה זמינה למשתמשים בכל עת.

- אבטחת מידע מקצה לקצה: המערכת נדרשת להגן על המידע שמועבר בה, כך שגם צד שלישי בעל גישה לרשת לא יוכל לקרוא את התוכן.
- פשטות ונוחות למשתמש: הממשק למשתמש צריך להיות אינטואיטיבי וקל להבנה, ללא צורך בהכשרה מוקדמת.

9.2 מודולי המערכת

המערכת מורכבת ממספר תתי-מערכות (מודולים) אשר כל אחד מהם אחראי על תחום מסוים בפעילות הכללית:

- מודול ניהול משתמשים: אחראי על תהליך ההזדהות, יצירת משתמשים וניהול פרטי הגישה.

מאפיין	תיאור
שם המודול	ניהול משתמשים ואימות זהות
תיאור	אחראי לרישום, כניסה, יצירת מפתחות הצפנה ואימות דו-שלבי.
פונקציות עיקריות	- יצירת חשבון משתמש - כניסה עם אימות דו-שלבי (2FA) - ניהול מפתחות קריפטוגרפיים (ציבורי ופרטי) - ניהול פרופיל משתמש
טכנולוגיות רלוונטיות	Spring Boot, Vaadin, MongoDB, JWT, 2FA API, Bcrypt

מודל שיחות אחראי על ניהול תקשורת הטקסט בין משתמשים. המודול מטפל ביצירת שיחות חדשות, תיעוד ההודעות, תיוג הודעות כנקרא/לא נקרא, ושמירה על רצף שיח תקין בין צדדים.

מאפיין	תיאור
שם המודול	ניהול שיחות
תיאור	יוצר ומנהל שיחות פרטיות וקבוצתיות בין משתמשים.
פונקציות עיקריות	- יצירת שיחה (פרטית/קבוצתית) - הוספה והסרה של משתתפים - אחסון מזהי שיחות ומטא-נתונים
טכנולוגיות רלוונטיות	MongoDB, REST API, JSON, UUID Generator

מודל הצפנה/פענוח אחראי על אבטחת המידע בתהליך ההעברה באמצעות הצפנה מקצה לקצה. המודול מיישם את מנגנוני ההצפנה (כגון ECDH ליצירת מפתחות ו AES-256 להצפנת התוכן), (וכן את תהליך הפענוח בצד המקבל).

מאפיין		תיאור
שם המודול	הצפנה ופענוח מידע	
תיאור	אחראי להצפנת ופענוח הודעות וקבצים לפי מפתח משותף מוצפן.	
פונקציות עיקריות	- החלפת מפתחות באמצעות ECDH - הצפנת תוכן עם AES-256 - יצירת IV ייחודי - חתימה דיגיטלית באמצעות ECDSA	
טכנולוגיות רלוונטיות	Java Cryptography API, BouncyCastle, AES, ECDH, ECDSA, SHA-256	

מודל שיתוף קבצים מאפשר שליחה וקבלה של קבצים בין משתמשים בצורה מאובטחת. כולל טיפול בהעלאת קבצים, הצפנתם, שליחתם, ואימות תקינותם בצד המקבל, תוך שמירה על פרטיות הנתונים.

מאפיין		תיאור
שם המודול	שיתוף קבצים מאובטח	
תיאור	מאפשר למשתמשים להעביר קבצים מוצפנים בצורה בטוחה.	
פונקציות עיקריות	- הצפנת קובץ לפי מפתח ייחודי - אחסון כתובת מוצפנת לקובץ - פענוח קבצים בצד הלקוח - הצגת סוג קובץ וסטטוס שליחה	
טכנולוגיות רלוונטיות	AES-256, Multipart Upload, MongoDB GridFS, File IO, Base64	

מודל סנכרון בזמן אמת אחראי על עדכון מיידי של נתונים בין המשתמשים, כולל הופעת הודעות חדשות, שינויי סטטוס, ואירועים אחרים במערכת. מבטיח חוויית שימוש רציפה ודינמית ללא צורך ברענון ידני של הממשק.

מאפיין	תיאור
שם המודול	תקשורת בזמן אמת
תיאור	מאפשר קבלת ושליחת הודעות בזמן אמת בין משתמשים.
פונקציות עיקריות	- פתיחת חיבור WebSocket - שליחת הודעות מיידיות - קבלת התראות ועדכונים - שמירה על חיבור יציב
טכנולוגיות רלוונטיות	WebSocket, STOMP, Spring Messaging, JSON

9.3 אפיון פונקציונלי וביצועים עקריים

המערכת נדרשת למלא את הפונקציות המרכזיות הבאות:

- הזדהות משתמשים: המערכת תאפשר למשתמשים להיכנס בצורה מאובטחת ולזהות זה את זה באופן חד-משמעי.
- העברת מסרים מאובטחת: כל מסר שנשלח במערכת יעבור תהליך של הגנה מפני יירוט או שינוי בדרך.
- מניעת התחזות: המערכת תספק כלים לוודא שמי ששולח את ההודעה הוא אכן מי שהוא טוען שהוא.
- שמירה על שלמות המידע: המערכת תוכל לזהות אם מסר שונה או הושחת.
- תגובה מיידי: המערכת תספק חוויית שימוש חלקה וללא עיכובים ניכרים בהעברת מידע.

9.4 אילוצי המערכת

- אילוצי חומרה: המערכת מיועדת לפעול על גבי מגוון מכשירים, כולל מחשבים שולחניים ומכשירים ניידים, ולכן עליה להיות מותאמת לפעולה בתנאים משתנים של משאבים (מעבד, זיכרון, רשת).
- אילוצי אבטחה: כל רכיב במערכת חייב לעמוד בדרישות מחמירות של הגנה על פרטיות, מניעת גישה לא מורשית, והתמודדות עם תקיפות נפוצות.
- אילוצי זמן תגובה: המערכת צריכה להבטיח זמן תגובה מהיר בכל שלב של השימוש, גם בתנאים של עומס.
- אילוצי תאימות: המערכת צריכה לפעול על פני מערכות הפעלה ודפדפנים שונים, תוך שמירה על אחידות בפונקציונליות ובמראה.
- אילוצי רגולציה ופרטיות: המערכת צריכה לעמוד בתקנים ודרישות חוקיות להגנה על מידע אישי, במיוחד אם תיושם בסביבה מקצועית או מוסדית.

10 תיאור התקיפה/ההגנה

בפרק זה נבחנים איומים קריפטוגרפיים עיקריים שעלולים לפגוע באבטחת התקשורת הדיגיטלית, כמו התקפות MitM, כוח גס והתחזות. מוצגים מנגנוני ההגנה של מערכת SmallTalk, בהם שימוש באלגוריתמים ECDH, AES-256 ו-ECDSA. בנוסף, מוסבר כיצד פועלים תהליכי ההצפנה, חתימה דיגיטלית ופענוח להבטחת פרטיות ושלמות המידע.

1. התקפת Man-in-the-Middle (MitM)

- **תיאור התקיפה:** תוקף מנסה להתמקם בין שני משתמשים כדי ליירט ולשנות את התקשורת ביניהם.
- **מנגנון הגנה:** מערכת SmallTalk מיישמת פרוטוקול ECDH להחלפת מפתחות שמונע מהתוקף לחשב את המפתח המשותף גם אם הוא מצליח ליירט את המפתחות הציבוריים. בנוסף, מימוש חתימות דיגיטליות ECDSA מאפשר לזהות שינויים בהודעות.

2. התקפות כוח גס על מפתחות הצפנה

- **תיאור התקיפה:** תוקף מנסה לפצח את ההצפנה באמצעות בדיקה שיטתית של כל המפתחות האפשריים.
- **מנגנון הגנה:** מערכת SmallTalk משתמשת בהצפנת AES-256 עם מפתחות באורך 256 סיביות, מה שהופך התקפת כוח גס לבלתי אפשרית מבחינה חישובית עם הטכנולוגיה הקיימת כיום.

3. התקפת שחזור מפתחות

- **תיאור התקיפה:** ניסיון לשחזר מפתחות פרטיים מתוך מידע ציבורי.
- **מנגנון הגנה:** המערכת מתבססת על הקושי המתמטי של בעיית הלוגריתם הדיסקרטי בעקומות אליפטיות, שהופכת את שחזור המפתח הפרטי מהמפתח הציבורי לבלתי אפשרי מבחינה חישובית.

4. התקפות Replay

- **תיאור התקיפה:** תוקף מקליט הודעות מוצפנות ושולח אותן מחדש בשלב מאוחר יותר.
- **מנגנון הגנה:** המערכת משתמשת בוקטורי אתחול (IV) ייחודיים לכל הודעה, מה שמבטיח שגם אם תוכן ההודעה זהה, ההצפנה תהיה שונה בכל פעם.

5. התקפות על שרת האחסון

- **תיאור התקיפה:** ניסיון לגישה למידע רגיש המאוחסן בשרתי המערכת.
- **מנגנון הגנה:** תודות להצפנת End-to-End, גם אם תוקף משיג גישה לשרתי המערכת, הוא לא יוכל לפענח את תוכן ההודעות ללא המפתחות הפרטיים של המשתמשים.

6. התקפות התחזות (Impersonation)

- **תיאור התקיפה:** תוקף מנסה להתחזות למשתמש לגיטימי כדי לתקשר עם משתמשים אחרים.
- **מנגנון הגנה:** המערכת מיישמת אימות דו-שלבי (2FA) ומערכת חתימות דיגיטליות המאפשרות למקבל לאמת את זהות השולח באמצעות המפתח הציבורי שלו.

מנגנוני אבטחה והגנה נוספים:

1. פרוטוקולי TLS/SSL

- מבטיחים תקשורת מאובטחת בין הלקוח לשרת, כשכבת הגנה נוספת מעבר להצפנת End-to-End.

2. מערכת גיבוי ושחזור מאובטחת

- גיבויים מוצפנים באמצעות AES-256
- שימוש בחתימות דיגיטליות לאימות שלמות הגיבויים
- מחיקה מאובטחת של נתונים שפג תוקפם

3. ניהול מפתחות מאובטח

- אחסון מאובטח של מפתחות פרטיים
- מנגנוני החלפת מפתחות מבוססי ECDH
- תהליכי גזירת מפתחות (Key Derivation) להפקת מפתחות הצפנה סופיים

4. אימות זהות רב-שכבתי

- אימות דו-שלבי (2FA)
- חתימות דיגיטליות לאימות זהות השולח
- מנגנוני בדיקת שלמות ההודעות

10.2 תיאור הצפנות

מנגנוני הצפנה מרכזיים:

1.

- **ECDH (Elliptic Curve Diffie-Hellman)** אלגוריתם להחלפת מפתחות מאובטחת המבוסס על עקומות אליפטיות.

- הפרוטוקול מאפשר לשני משתתפים שלא נפגשו מעולם ואינם חולקים ביניהם סוד משותף כלשהו מראש, להעביר אחד לשני מעל גבי ערוץ פתוח (שאינו מאובטח) סוד כלשהו כך שאיש מלבדם אינו יודע מהו. הסוד יכול להיות, בהקשר של פרוטוקול תקשורת מאובטח, מפתח הצפנה. פרוטוקול דיפי-הלמן מתמודד עם בעיית הפצת המפתחות על ידי פונקציה חד-כיוונית.

• תהליך :

- כל משתמש יוצר זוג מפתחות: מפתח פרטי (d) ומפתח ציבורי, $(Q = d \times G)$, כאשר G היא נקודת בסיס על העקומה האליפטית
- המשתמשים מחליפים ביניהם רק את המפתחות הציבוריים
- כל משתמש מחשב את המפתח המשותף $K = d \times Q_{\text{other}}$: מפתח פרטי שלו כפול המפתח הציבורי של הצד השני)

- **יתרונות:** מספק אבטחה גבוהה עם מפתחות קצרים יותר בהשוואה ל-RSA או DH רגיל, מה שמספר את הביצועים תוך שמירה על רמת אבטחה גבוהה.

2. (אלגוריתם סימטרי להצפנת הודעות, המספק רמת אבטחה גבוהה במיוחד הצפנת-AES

256 היא דרך לשמור על הודעות סודיות או מידע בטוח מאנשים שלא אמורים להיות מסוגלים לראות אותם. הצפנת AES-256 היא כמו שיש לך מנעול סופר חזק על הקופסה שלך שניתן לפתוח רק על ידי מפתח מאוד ספציפי. המנעול כל כך חזק שיהיה קשה מאוד למישהו לשבור אותו ולפתוח את הקופסה ללא המפתח הנכון.

יצירת מפתחות (Key Generation)

כל משתמש בגרסה של אלגוריתם הצפנה מבוסס על קריפטוגרפיה של עקומות אליפטיות (Elliptic Curve Cryptography, ECC) יוצר זוג מפתחות:

• מפתח פרטי (Private Key)

המפתח הפרטי הוא למעשה "המפתח הסודי" של המשתמש. הוא מיוצר כמספר אקראי בעל אורך קבוע (לדוגמה 256 סיביות), והוא צריך להישמר בסודיות מוחלטת. דמיינו את המפתח הפרטי כמו מפתח של כספת אישית - אף אחד לא צריך לדעת עליו חוץ מבעל הכספת עצמו.

• מפתח ציבורי (Public Key)

המפתח הציבורי נוצר על ידי ביצוע פעולה מתמטית על המפתח הפרטי באמצעות העקומה האליפטית. הנוסחה המתמטית $d \times G$ יוצרת את המפתח הציבורי Q , כאשר:

- d הוא המפתח הפרטי שמקבל מספר אקראי גדול ביותר שנבחר בטווח מסוים.
- G היא נקודת הבסיס על העקומה האליפטית נקודת הבסיס והיא נקודה קבועה ומוגדרת מראש על העקומה האליפטית. היא משמשת כ"נקודת התחלה" לחישוב המפתח הציבורי.
- Q הוא המפתח הציבורי והוא למעשה נקודה גיאומטרית על העקומה האליפטית, המתקבלת על ידי פעולת הכפל.

דוגמה להמחשה

נניח שיש לנו מפתח פרטי $d = 6$, ונקודת בסיס G על העקומה. נבצע את החישוב: $Q = 6 \times G$ שהוא המפתח הציבורי.

החלפת מפתחות (Key Exchange)

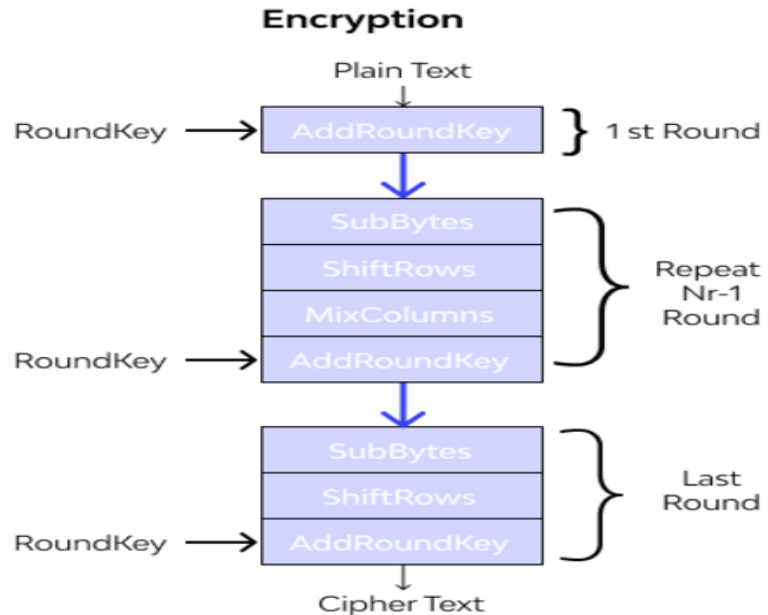
בשלב זה, שני משתמשים A ו B רוצים להסכים על מפתח משותף שמאפשר הצפנה ופענוח של הודעות ביניהם, מבלי שהם ישתפו במפתח הפרטי שלהם.

- כל אחד מהמשתמשים שולח את המפתח הציבורי שלו לשני הצדדים.
- משתמש A מחשב את המפתח המשותף K באמצעות המפתח הפרטי שלו A^d
- והמפתח הציבורי של משתמש B B^Q

$$B^Q \times A^d = K$$

משתמש B מחשב את המפתח המשותף K בעזרת המפתח הפרטי B^Q שלו והמפתח הציבורי של משתמש A A^d .

הצפנת הודעות (Encryption)



פה בתמונה ניתן לראות איך ההצפנה פועלת

לאחר שהשניים הסכימו על המפתח המשותף K הם יכולים להשתמש בו כדי להצפין את ההודעות:

- המפתח המשותף K עובר בדרך כלל תהליך של גזירת מפתח (Key Derivation) על

מנת להפיק מפתח חזק יותר וייחודי להצפנה. לדוגמה, ניתן להשתמש בפונקציות Hash

כמו SHA-256 על מנת ליצור את המפתח הסופי $Final^K$.

$$SHA-256(K) = \text{final}K$$

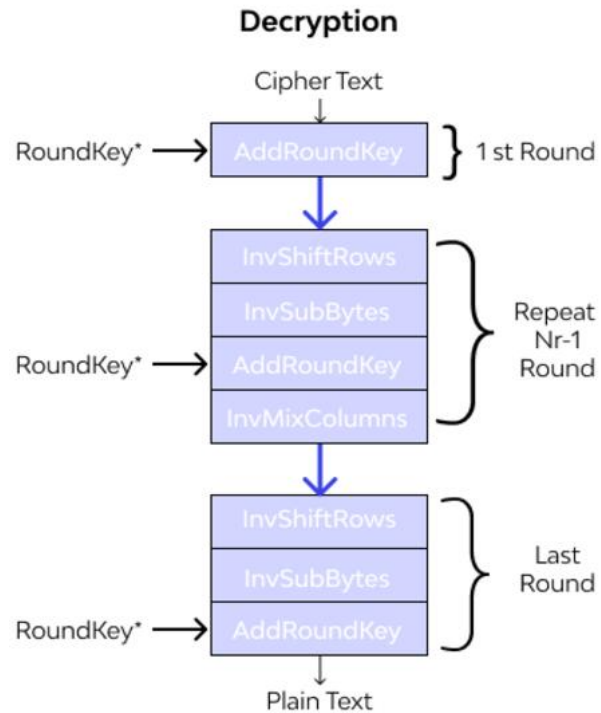
הודעה M מוצפנת באמצעות אלגוריתם AES-256 אלגוריתם הצפנה סימטרי:

$$\text{Encrypt}_{\text{final}K}(M) = C$$

בכדי למנוע התקפות הצפנה חוזרת, מצורף לנתונים וקטור אתחול (IV) לכל הודעה, מה שהופך את הצפנת ההודעה לאי-חזרתית.

וקטור אתחול (Initialization vector - IV) הוא מחרוזת סיביות ייחודית כלשהי, באורך מוגדר המשמשת להתחלת תהליך ההצפנה בצופן זרם או בצופן בלוקים כאשר עושים שימוש במצב הצפנה מסוג שרשור (Chaining).

פענוח הודעות (Decryption):



בתמונה פה ניתן לראות את הפענוח

- לאחר קבלת ההודעה המוצפנת, המקבל יכול לפענח אותה כך:

הוא מחשב את המפתח המשותף K בעזרת המפתח הפרטי שלו $receiver^d$ והמפתח הציבורי של השולח $sender^Q$

$$sender^Q \times receiver^d = K$$

לאחר מכן, הוא גוזר את המפתח הסופי $Final^K$ ומשתמש בו לפענוח ההודעה:

$$Decrypt_{final^K}(C) = M$$

ההודעה המפוענחת M היא ההודעה המקורית שנשלחה על ידי השולח.

3. ECDSA (Elliptic Curve Digital Signature Algorithm) לחתימות דיגיטליות

- **תיאור:** אלגוריתם לחתימה דיגיטלית המבוסס על עקומות אליפטיות, המאפשר לאמת את מקור ושלמות ההודעה.
- **תהליך:**
 - השולח חותם על ההודעה באמצעות המפתח הפרטי שלו
 - החתימה הדיגיטלית נשלחת יחד עם ההודעה המוצפנת
 - המקבל מאמת את החתימה באמצעות המפתח הציבורי של השולח
- **יתרונות:** מבטיח את אותנטיות ההודעה, מונע התכחשות (non-repudiation) ומאפשר לזהות שינויים בהודעה במהלך העברתה.

תהליך הצפנה מלא:

1. יצירת מפתחות:

- כל משתמש מייצר זוג מפתחות ECDH (פרטי וציבורי)
- המפתח הפרטי נשמר במכשיר המשתמש בלבד, המפתח הציבורי משותף עם השרת

2. החלפת מפתחות:

- כאשר משתמש A רוצה לתקשר עם משתמש B, המערכת מבצעת החלפת מפתחות ECDH
- מחושב מפתח משותף ייחודי K לשני המשתמשים

3. הצפנת הודעות:

- המפתח המשותף K עובר גזירת מפתח ליצירת מפתח הצפנה סופי K_Final
- לכל הודעה נוצר IV ייחודי
- ההודעה מוצפנת באמצעות AES-256 עם המפתח הסופי והוקטור
- נוצרת חתימה דיגיטלית על ההודעה המוצפנת באמצעות ECDSA
- ההודעה המוצפנת, ה-IV, והחתימה הדיגיטלית נשלחים לשרת

4. פענוח הודעות:

- המקבל מאמת את החתימה הדיגיטלית באמצעות המפתח הציבורי של השולח
- אם החתימה תקינה, המקבל משתמש במפתח המשותף K_Final שלו לפענוח ההודעה באמצעות AES-256

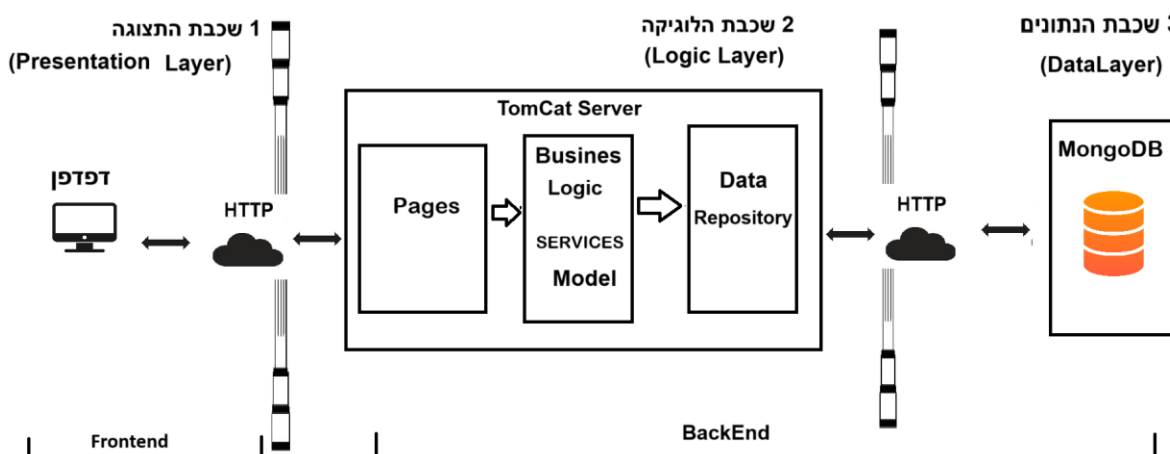
תהליכי האבטחה המשולבים במערכת SmallTalk מבטיחים פרטיות מלאה ואבטחת מידע ברמה גבוהה, כאשר גם שרתי המערכת אינם יכולים לקרוא את תוכן ההודעות המוצפנות - מה שמבטיח הצפנת End-to-End אמיתית.

11 תיכון מערכת

בפרק זה מתואר תכנון המערכת SmallTalk תוך שימוש בארכיטקטורת שלוש שכבות הכוללת ממשק משתמש, לוגיקה עסקית ושכבת נתונים. בנוסף מוסבר מודל השרת-לקוח, מבנה מסד הנתונים מבוסס MongoDB, תהליכי המערכת ופרוטוקולי התקשורת WebSocket ו-HTTP. הפרק מדגיש את שילוב האבטחה וההצפנה בכל שכבות המערכת.

11.1 ארכיטקטורת המערכת (ע"פ מודל 3 השכבות)

ארכיטקטורת שלוש השכבות (Three-Tier Architecture) היא מודל נפוץ לפיתוח מערכות תוכנה, המפריד את המערכת לשלושה חלקים עיקריים שכבת התצוגה, שכבת הלוגיקה ושכבת הנתונים. לכל שכבה בארכיטקטורה זו יש תפקיד ברור, והיא מתמקדת בהיבט אחר של המערכת. חלוקה זו נועדה לשפר את תחזוקת המערכת, לאפשר גמישות בשדרוגים ולייעל את העבודה בצוותי פיתוח. ולהלן תרשים שלושת השכבות



התמונה מציגה ארכיטקטורת תלת-שכבתית:

שכבת התצוגה (Presentation Layer) שהיא ה- Frontend המשתמש בדפדפן כדי להתחבר למערכת ב-Backend באמצעות פרוטוקול HTTP כדי להציג את הדפים ובניהם דף הבית. **שכבת הלוגיקה (Logic Layer)** כוללת את הצד האחורי (Backend) עם שרת TomCat, לוגיקת עסקים (Business Logic) ויש גם services ומאגר נתונים (Data Repository), ושכבת הנתונים (Data Layer) מכילה את בסיס הנתונים (Database) עם חיבור HTTP למאגר הנתונים.

11.2 תיוכון מפורט של רכיבי המערכת

הסבר על התרשים שלושת השכבות

1. שכבת ממשק המשתמש (Presentation Layer)

- אחראית על אספקת ממשק משתמש מאובטח ונוח למשתמשים.
- ככל הנראה נבנית באמצעות טכנולוגיה מבוססת אינטרנט, כמו פלטפורמת Vaadin המאפשרת בניית ממשקים עשירים ותגובתיים בשפת Java.
- הממשק מתקשר עם שכבת הלוגיקה העסקית בשני אופנים:
- דרך ממשק REST API לביצוע פעולות כמו התחברות, רישום וניהול פרופיל.
- דרך חיבור WebSocket לצורך תקשורת בזמן אמת, כמו הודעות מיידיות.
- כדי להבטיח הצפנה מקצה לקצה (E2EE) של נתוני המשתמשים, הממשק מיישם הצפנה באמצעות שילוב של ECDH להחלפת מפתחות ו AES-256 להצפנת ההודעות.

2. שכבת הלוגיקה (Logic Layer):

- שרת TomCat אחראי על הלוגיקה העסקית המרכזית ועל ניהול הנתונים.
- חושפת מערך של ממשקי REST API שהממשק יכול לצרוך.
- מחולקת לשלושה רכיבים עיקריים:
- שירותים (Services) מכילים את הלוגיקה העסקית העיקרית, כמו ניהול משתמשים, הודעות, שיתוף קבצים, והצפנה/פענוח.
- מאגרים (Repositories) מספקים הפשטה לאחסון ואחזור הנתונים, כנראה באמצעות מסד נתונים NoSQL כמו MongoDB.
- משלבת מנגנוני אבטחה כמו אימות, הרשאות, והצפנה, להגנה על האפליקציה ונתוני המשתמשים.
- ככל הנראה מיושמת באמצעות פריימוורק Java כמו Spring Boot.

3. שכבת הנתונים (Data Layer)

- אחראית על אחסון ואחזור של נתוני המשתמשים, היסטוריית ההודעות, ומטא-נתוני הקבצים.
 - מחוברת לשכבת הלוגיקה העסקית ומספקת שירותי אחסון ואחזור נתונים.
 - בהתאם לשימוש במסד נתונים NoSQL כמו MongoDB, יכולה לטפל ביעילות בדרישות הנתונים המשתנות של אפליקציית SmallTalk.
- הפרדה לשכבות נפרדות מקדמת תחזוקה של האפליקציה. זה גם מאפשר להתמקד בחזקתה על אבטחה ופרטיות, כאשר המנגנוני הצפנה משולבים בתוך האדריכלות.

הבחירה בטכנולוגיות כמו Vaadin ו Spring Boot מבטיחה עקביות טכנולוגית ומקלה על האינטגרציה בין רכיבי הלקוח והשרת.

11.3 מודל שרת/לקוח

המערכת SmallTalk פועלת לפי **מודל שרת-לקוח (Client-Server Model)** שהוא מודל ארכיטקטוני קלאסי בפיתוח מערכות תקשורת. במודל זה, יש הפרדה ברורה בין שני צדדים:

- **הלקוח (Client)** – האפליקציה שרצה בדפדפן של המשתמש.

- **השרת (Server)** – האחראי על ניהול הנתונים, האבטחה והלוגיקה העסקית.

עקרונות המודל:

- הלקוח שולח בקשות לשרת (למשל: התחברות, שליחת הודעה).
- השרת מקבל את הבקשה, מעבד אותה, ניגש למסד הנתונים (אם צריך), ומחזיר תגובה ללקוח.
- הלקוח מציג את התוצאה למשתמש.

אופן הפעולה במערכת SmallTalk

במערכת משולבים שני סוגים של תקשורת בין הלקוח לשרת:

1. תקשורת באמצעות REST API

- **שימוש עיקרי:** פעולות בסיסיות (התחברות, רישום, שליפת היסטוריית שיחות).
- **פרוטוקול:** HTTP
- **פורמט נתונים:** JSON
- **אופן פעולה:**
 - המשתמש ממלא טופס התחברות → הלקוח שולח בקשת POST לשרת עם הנתונים.
 - השרת בודק את הנתונים מול מסד הנתונים (MongoDB), ואם הכל תקין – מחזיר תגובה עם פרטי המשתמש.
 - הלקוח מציג את המידע על המסך.

2. תקשורת בזמן אמת באמצעות WebSocket

- **שימוש עיקרי:** שליחת הודעות מיידיות, קבלת התראות, סנכרון בזמן אמת.
- **מאפיינים:**
 - פתיחת חיבור קבוע (persistent connection) בין הלקוח לשרת.
 - תקשורת **דו-כיוונית** – גם השרת יכול לפנות ללקוח בזמן אמת.

- **אופן פעולה:**

- בעת טעינת האפליקציה, הלקוח פותח חיבור WebSocket מול השרת.
- כאשר משתמש שולח הודעה → ההודעה מוצפנת (AES) ונשלחת לשרת דרך ה-WebSocket.
- השרת מקבל את ההודעה, שולח אותה למקבל בזמן אמת – גם כן דרך WebSocket.
- אצל המקבל: ההודעה מתקבלת, מפוענחת ומוצגת באופן מיידי.

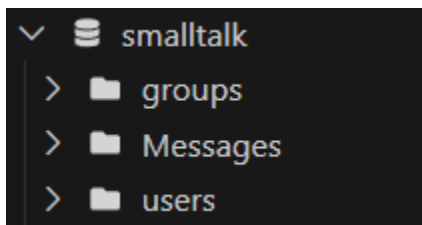
- **יתרונות המודל:**

- **הפרדה ברורה בין שכבות** – ממשיק משתמש, לוגיקה עסקית ונתונים.
- **גמישות** – ניתן להחליף/לשדרג את צד הלקוח או השרת בלי לפגוע במערכת כולה.
- **תמיכה בעדכון בזמן אמת** – שילוב REST ו WebSocket-מאפשר גם פעולה רגילה וגם תקשורת חיה.

11.4 תיאור מסד הנתונים

בפרויקט השתמשתי במסד נתונים MongoDB - מסד נתונים בענן המבוסס אוספים של מסמכים. מאפיינים טכנולוגיים: מסד נתונים NoSQL – כלומר, כל המידע/הנתונים נשמרים במבנה של אובייקטי JSON, אחסון מסמכים גמיש, ביצועים גבוהים, הרחבה קלה, תמיכה מלאה בהצפנה.

להלן האוספים הקיימים במסד הנתונים של המערכת:



1. אוסף משתמשים Users Collection

תפקיד : ניהול זהות משתמשים ואימות כניסה

שדות המשתמש:

- **_id** (מחרוזת) כתובת דוא"ל של המשתמש (משמשת כמזהה ייחודי)
 - **fullName** (מחרוזת) שם מלא של המשתמש
 - **password** (מחרוזת) סיסמה (בפועל אמורה להיות מוצפנת)
 - **_class** (אובייקט) מזהה את סוג הישות בקוד
- דוגמה למשתמש יחיד, כפי שהוא שמור במסד הנתונים:

```
_id: "michaell@gmail.com"
fullName: "michael uliel"
password: "2222"
_class: "mu.smalltalk.entitis.User"
```

2. אוסף קבוצות Groups Collection

תפקיד : ייצוג קבוצות תקשורת בין משתמשים

שדות הקבוצה :

- **_id** (מחרוזת) מזהה ייחודי לקבוצה למשל ("group1")
- **users** (מערך מחרוזות) תרשימת כתובות אימייל של המשתמשים החברים בקבוצה
- **_class** (אובייקט) מזהה את סוג היישות בקוד

```
_id: "group1"
▼ users : Array (2)
  0: "ilanp@gmail.com"
  1: "michaell@gmail.com"
_class : "mu.smalltalk.entitis.Group"
```

```
▶ _id: "group2"
▼ users : Array (4)
  0: "ilanp@gmail.com"
  1: "danieli@gmail.com"
  2: "elichayo@gmail.com"
  3: "michaell@gmail.com"
_class : "mu.smalltalk.entitis.Group"
```


3. Messages Collection

תפקיד : אחסון הודעות מוצפנות בין משתמשים, כולל פרטי השולח, תוכן ההודעה המוצפן, חתימה דיגיטלית וזמן השליחה.

שדות ההודעה:

- **_id** (אובייקט) מזהה ייחודי להודעה במסד הנתונים.
- **groupId** (מחרוזת) מזהה של הקבוצה או השיחה שאליה שייכת ההודעה לדוגמה ("group1")
- **sender** (מחרוזת) כתובת הדוא"ל של השולח לדוגמה ("michael@gmail.com")
- **encryptedContent** (מחרוזת) תוכן ההודעה לאחר הצפנה, מקודד בפורמט Base64 שמור כ- Binary ב-MongoDB)
- **timestamp** (תאריך ושעה) זמן שליחת ההודעה.
- **_class** (אובייקט) מזהה את סוג היישות בקוד

```
{
  "_id": {
    "$oid": "6827556291ea3d568e676aee"
  },
  "senderID": "michaell@gmail.com",
  "receiverID": "group1",
  "textContent": {
    "$binary": {
      "base64": "WzIwMjUtMDUtMTYgMTg6MTA6MjZdIG1pY2hhZWwgdWxpZWw6PGJyPteU15nXmQ==",
      "subType": "00"
    }
  },
  "chatId": "group1",
  "time": {
    "$date": "2025-05-16T15:10:26.846Z"
  },
  "_class": "mu.smalltalk.entitis.Message"
}
```

11.5 תהליכים במערכת ההפעלה

במערכת SmallTalk, מערכת ההפעלה מפעילה תהליכים כמו חיבור למסד הנתונים (MongoDB) טיפול בבקשות משתמשים, והרצת אלגוריתמים קריפטוגרפיים כגון AES-256 ו- ECDH שירותים כמו ChatService ו- GlobalMessageBroadcaster עשויים לפעול כחוסים נפרדים (Threads) כדי לאפשר שליחה וקבלה של הודעות בזמן אמת תוך שמירה על תגובתיות גבוהה מערכת ההפעלה אחראית גם לניהול משאבים בין התהליכים, כמו הקצאת זיכרון ו- CPU.

11.6 תיאור פרוטוקולי תקשורת

תיאור פרוטוקולי תקשורת

• פרוטוקול WebSocket

קשר חי ומידי מספק חיבור דו-כיווני רציני בין הלקוח (דפדפן) לבין השרת. זה אומר שאם יש שינויים בנתונים בצד השרת, ניתן לשלוח את העדכונים ללקוח (client) בזמן אמת מבלי להפעיל בקשות חדשות.

• פרוטוקול HTTP

(HTTP (Hypertext Transfer Protocol הוא פרוטוקול המשמש לתקשורת בין שרת האינטרנט לדפדפן האינטרנט של המשתמש. הוא מאפשר לדפדפנים לבקש מידע משרתים ולהציג אותו למשתמשים. HTTP באתר שלי זה ישמש לתקשורת בין הלקוח והשרת.

11.7 שירותים חיצוניים (REST API)

מטרה: משפטי חיזוק דינמיים לעמוד הבית באמצעות שירות API חיצוני
השירות החיצוני בשימוש:

ZenQuotes API

- כתובת url <https://zenquotes.io/api/random>
- תפקיד: מקור ראשי למשפטי חיזוק באנגלית
- פורמט תגובה: מערך JSON עם שדות q(משפט) ו a-מחבר)

מימוש טכני:

QuoteService Class:

- שירות Spring המנהל את הקריאות ל API
- יוצר RestTemplate עם הגדרות SSL מותאמות אישית
- מטפל בשגיאות רשת ומספק חלופות

אינטגרציה בעמוד הבית:

- משפט חיזוק מוצג בבר הניווט העליון
- רענון אוטומטי כל 10 דקות
- כפתור רענון ידני עם אפקטים חזותיים
- fallback למשפטים מקומיים בעברית במקרה של כשל

מנגנון: Fallback

1. ניסיון ראשון - ZenQuotes API
2. גיבוי מקומי - מערך קבוע של 15 משפטי חיזוק בעברית
3. גיבוי נוסף - משפטים באנגלית מוגדרים מראש בQuoteService

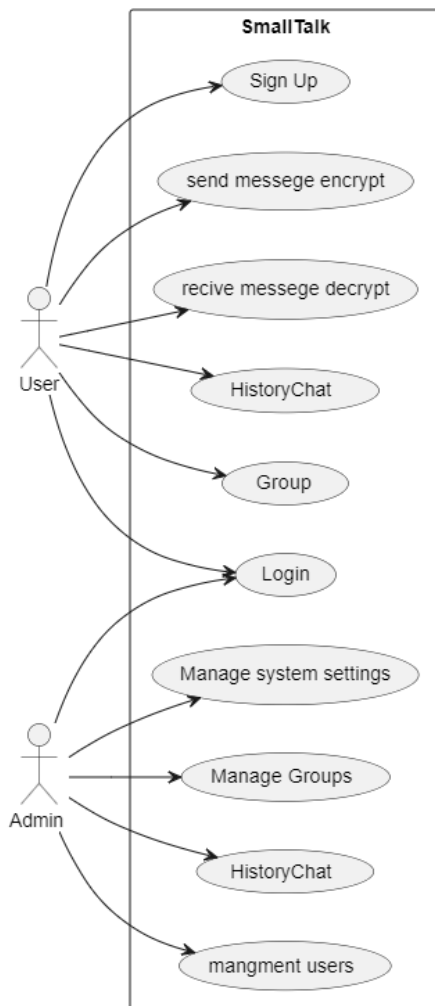
12 ניתוח תרחישים וזרימת המידע

בפרק זה ננתח את תרחישי השימוש העיקריים במערכת SmallTalk ונתאר את זרימת המידע במצבים שונים. נציג תרשימי Use-Case המדגים את פעולות המשתמשים, ותרשימי רצף הממחישים תהליכים מרכזיים כגון הרשמה, התחברות, שליחת הודעות וניהול קבוצות. מטרת הפרק היא להבין את ההתנהגות הדינמית של המערכת ואת אופן התקשורת בין רכיביה.

12.1 תרשימי תרחיש Use-Case Diagram

התרשים המוצג הוא תרשימי Use-Case (תרחישי שימוש) עבור מערכת בשם SmallTalk. התרשים מתאר את השחקנים (Actors) במערכת ואת הפעולות המרכזיות (Use Cases) שכל אחד מהם יכול לבצע.

- **User (משתמש רגיל):** משתמש רגיל של המערכת, יכול לבצע פעולות בסיסיות.
- **Admin (מנהל):** בעל הרשאות ניהול, יכול לבצע פעולות מתקדמות וניהוליות.



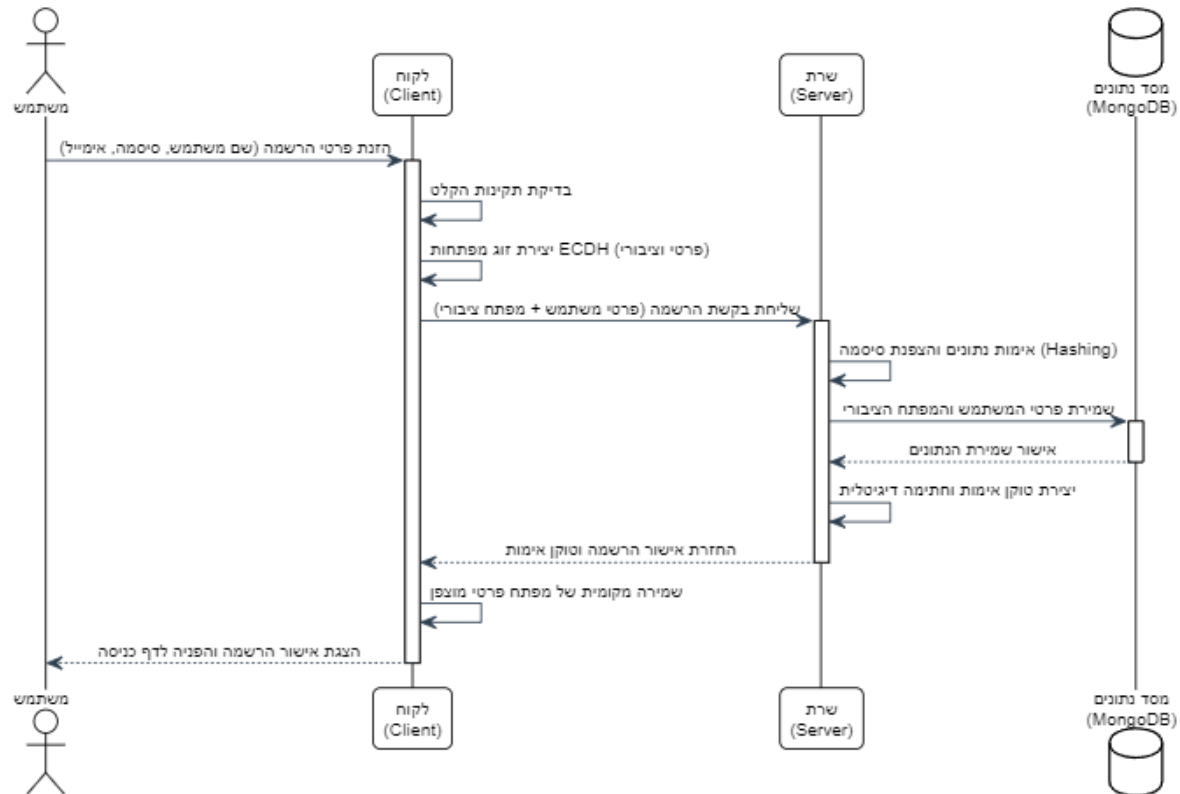
ולהלן הסבר על התרחישי שימוש

הסבר	שחקן מבצע	התרחש שימוש
הרשמה לאפליקציה. המשתמש ציץ תעביר חדש כדי להיכנס להשתמש בשירות.	User	Sign Up
שליחת הודעה מוצפנת. המשתמש שולח הודעה מוצפנת לצורך אבטחת הפרטיות.	User	send message encrypt
קבלת הודעה ופענוחה. המשתמש מקבל הודעה מוצפנת ומפענח אותה לקריאה.	User	receive message decrypt
צפייה בהיסטוריית הצ'אט. מאפשר למשתמשים לעיין בהודעות קודמות.	User, Admin	HistoryChat
כניסה למערכת באמצעות שם משתמש וסיסמה.	User	Login
יצירת קבוצה חדשה. המשתמש יוצר קבוצה לתקשורת עם מספר משתתפים.	User	Create Group
הצטרפות לקבוצה קיימת. המשתמש מתחבר לקבוצה שנוצרה על ידי משתמש אחר.	User	Join Group
שליחת הודעה לקבוצה. המשתמש שולח הודעה מוצפנת לכל חברי הקבוצה.	User	Send Group Message
עזיבת קבוצה. המשתמש יוצא מקבוצה ומפסיק לקבל הודעות ממנה.	User	Leave Group
צפייה בחברי הקבוצה. המשתמש רואה רשימה של כל המשתתפים בקבוצה.	User	View Group Members
ניהול משתמשים. המנהל יכול לחסום, למחוק או לערוך משתמשים במערכת.	Admin	management users
ניהול הגדרות מערכת. המנהל יכול לשנות הגדרות כלליות של המערכת.	Admin	Manage system settings
ניהול קבוצות במערכת. המנהל יכול לראות ולנהל את כל הקבוצות הקיימות.	Admin	Manage Groups
מחיקת קבוצה. המנהל יכול למחוק קבוצות לא רצויות או לא פעילות.	Admin	Delete Group
הוספה והסרה של חברי קבוצה. המנהל יכול לנהל את חברות הקבוצות.	Admin	Add/Remove Group Members

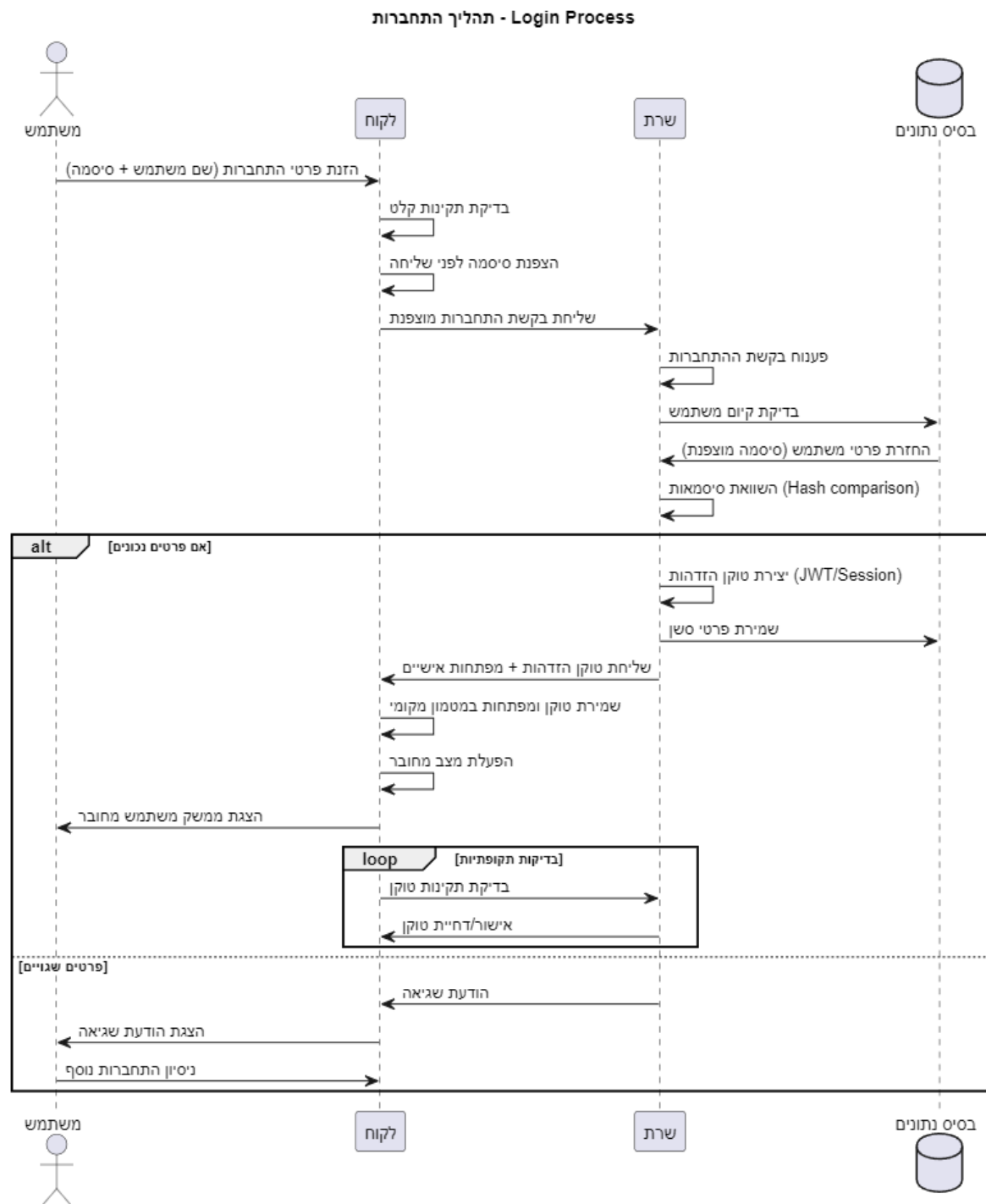
תרשימי רצף Sequence Diagram

12.2

תהליך הרשמה ויצירת מפתחות קריפטוגרפיים

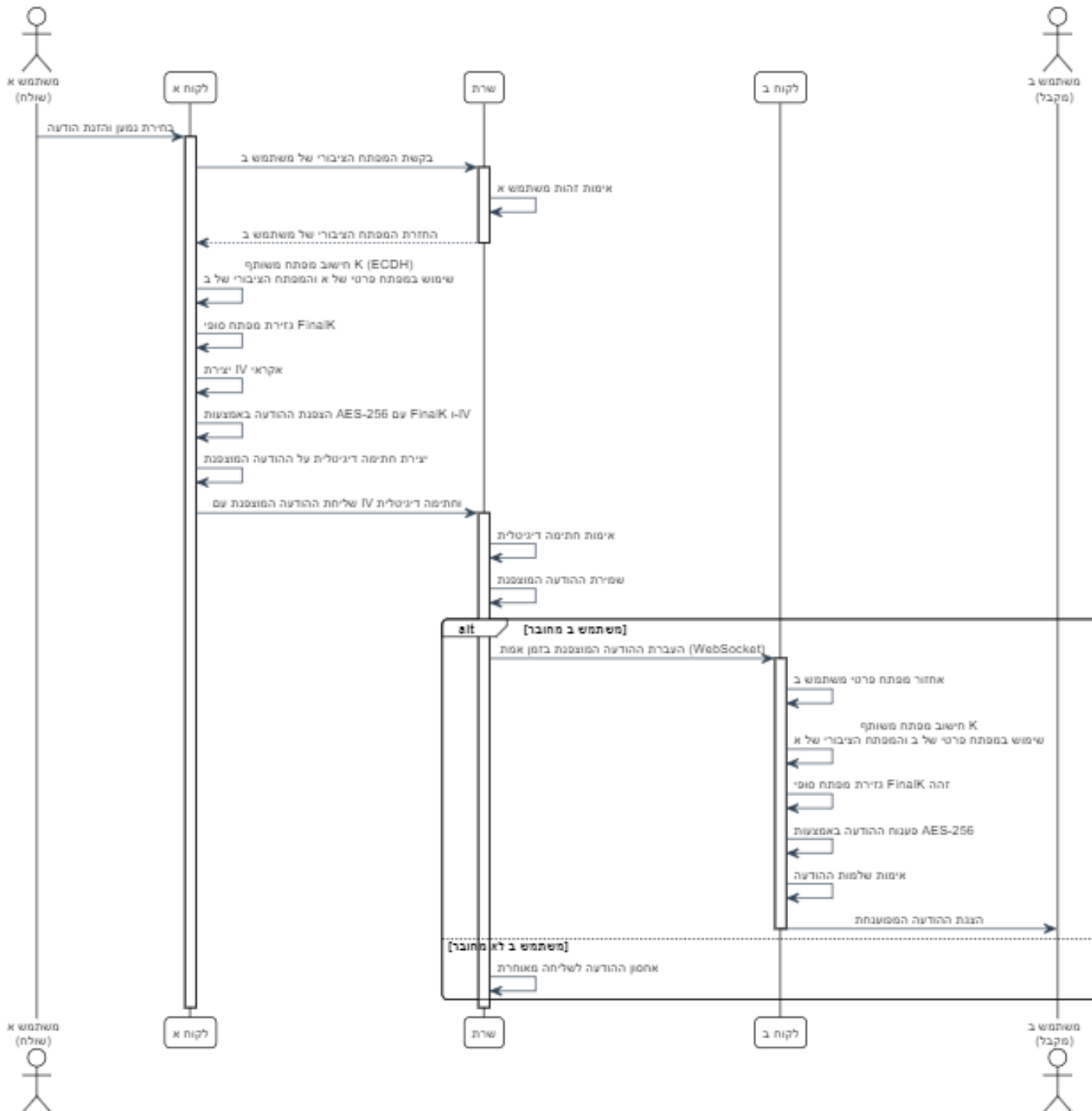


תרשים זה מתאר את תהליך ההרשמה הבטוח למערכת, החל מבקשת המשתמש לפתיחת חשבון, דרך אימות הנתונים ושמירתם בבסיס הנתונים המוצפן. התהליך כולל יצירת מפתחות קריפטוגרפיים ייחודיים עבור המשתמש ושמירתם בצורה מאובטחת.



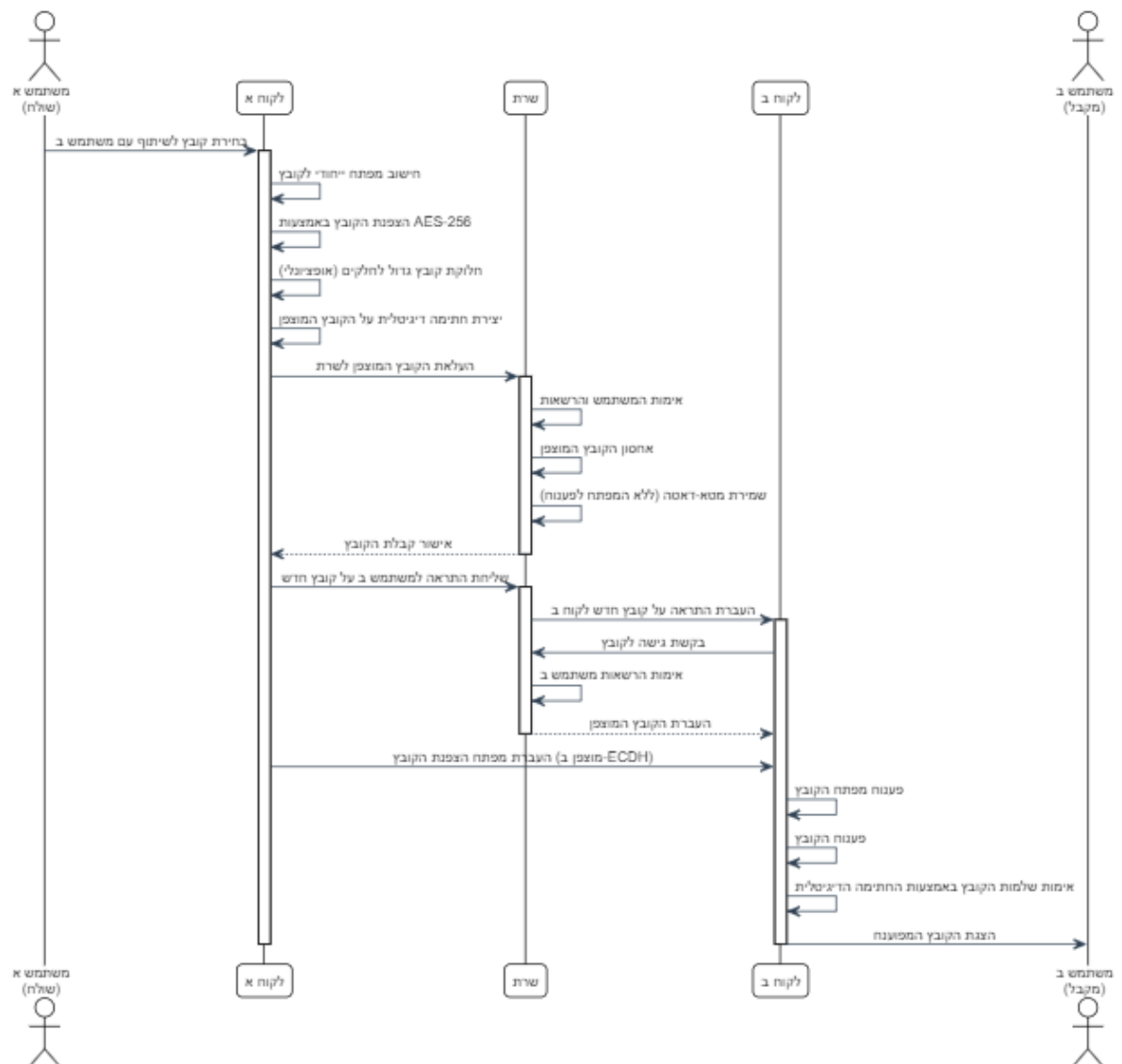
תרשים מפורט של תהליך האימות והתחברות, החל מהזנת פרטי ההתחברות דרך הצפנתם ושליחתם לשרת, אימות מול בסיס הנתונים, ויצירת טוקן הזדהות. התרשים כולל גם טיפול בשגיאות ובדיקות תקופתיות לתקינות הטוקן.

תהליך החלפת מפתחות ושליחת הודעה מוצפנת



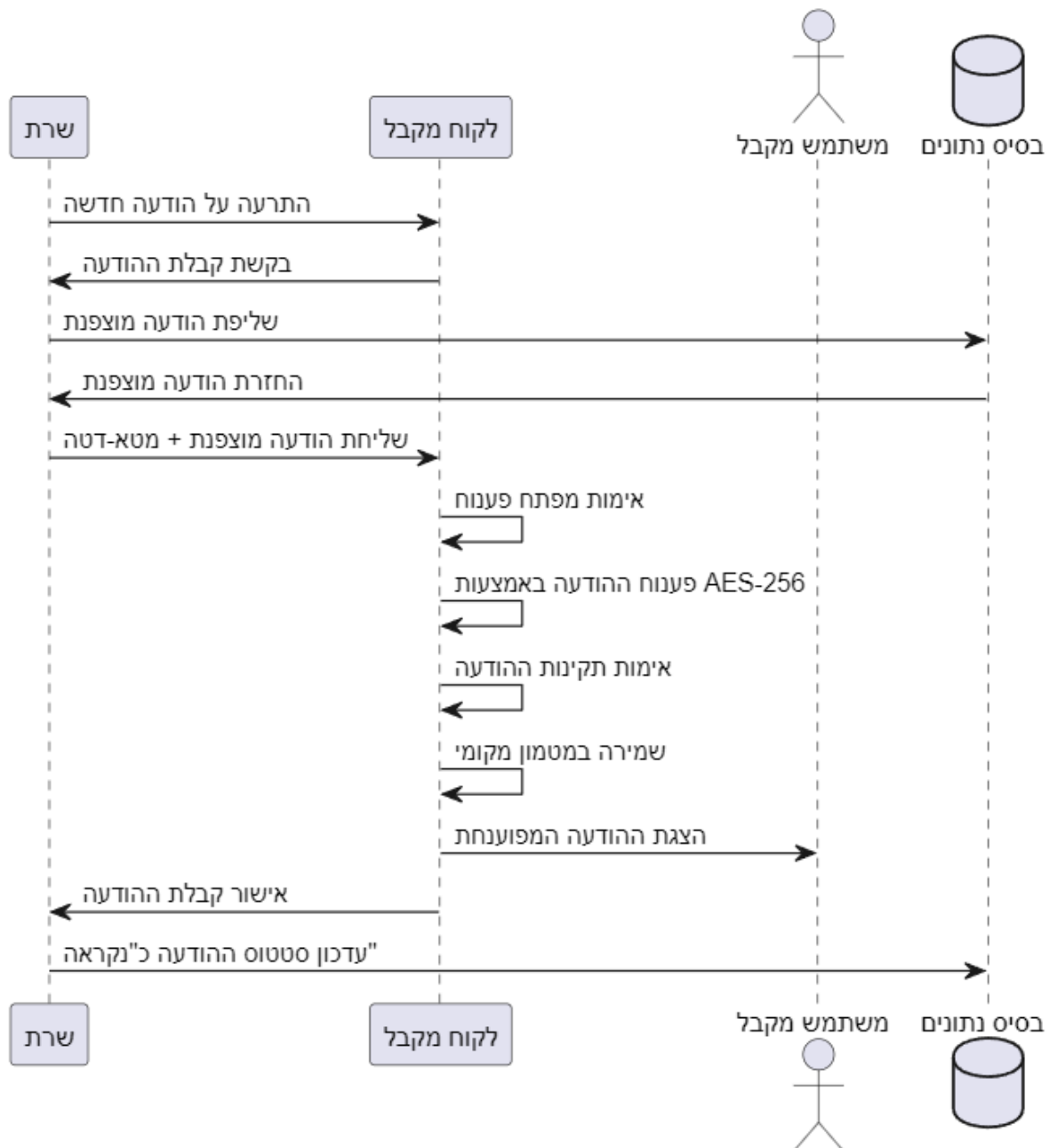
תרשים מורכב המציג את תהליך החלפת המפתחות הבטוח בין שני משתמשים באמצעות פרוטוקול ECDH יצירת מפתח סשן זמני, והצפנת ההודעה במפתח AES-256. התהליך מבטיח שליחה מאובטחת של הודעות עם הצפנה end-to-end.

תהליך שיתוף קבצים מוצפנים



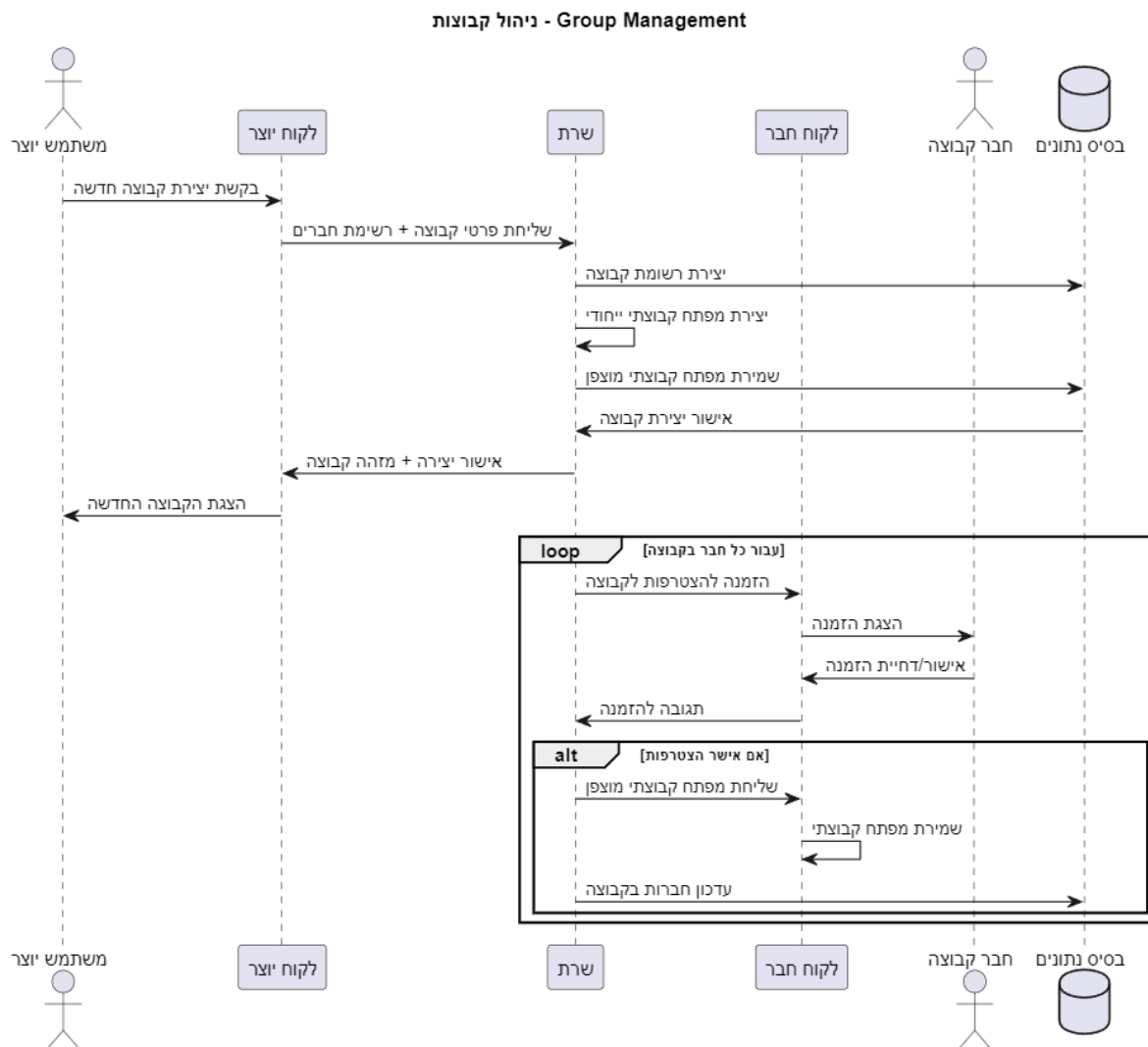
תרשים המתאר את מנגנון השליחה הקבוצתית המוצפנת, כולל יצירת מפתח קבוצתי משותף, הצפנת ההודעה עבור כל חברי הקבוצה, וחלוקה מאובטחת של המפתחות. המערכת מבטיחה שכל חברי הקבוצה יוכלו לפענח את ההודעות בצורה מאובטחת.

קבלת הודעה מוצפנת - Receive Encrypted Message



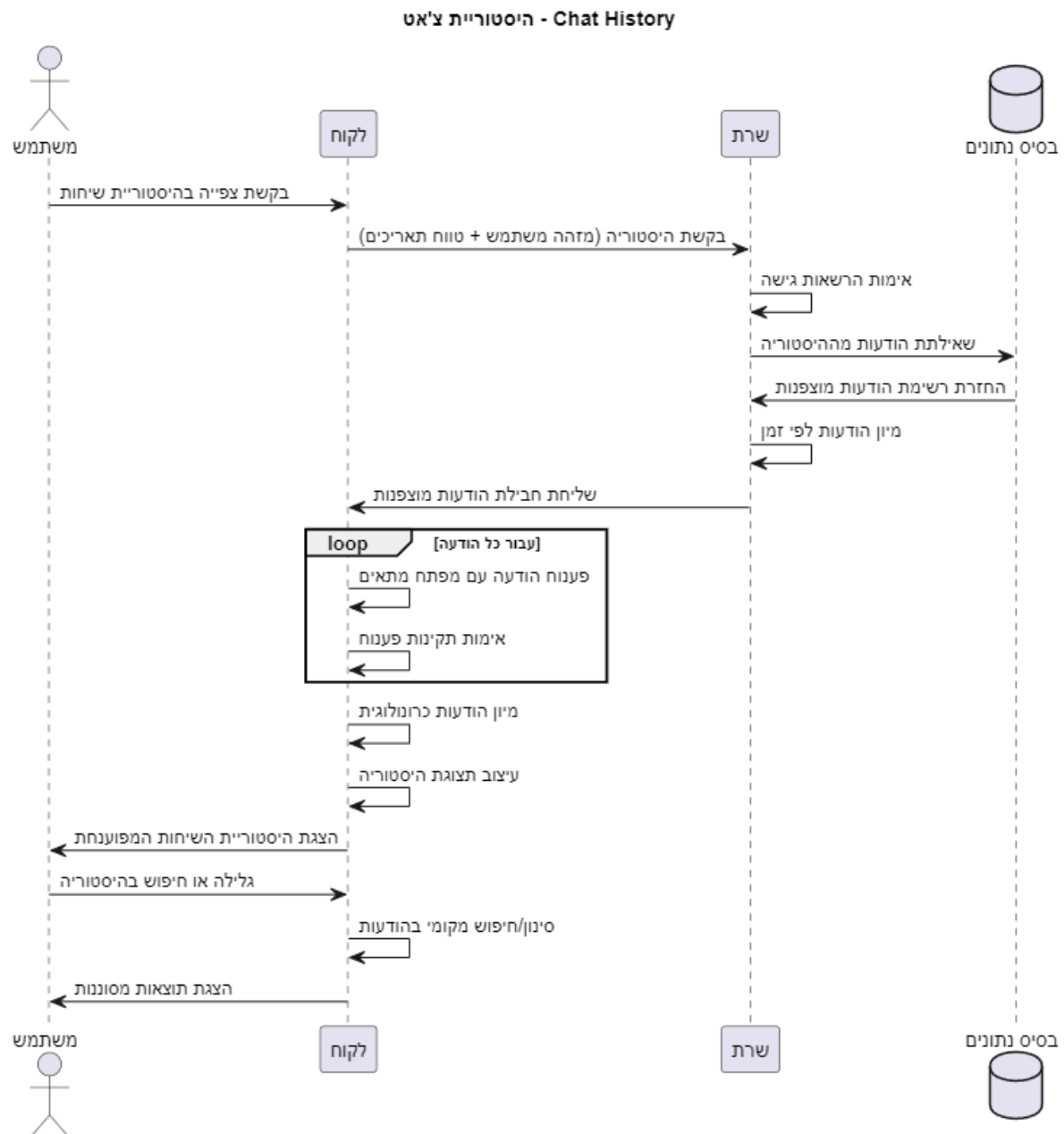
קבלת הודעה מוצפנת (Receive Encrypted Message)

תרשים זה מתאר את התהליך המלא של קבלת הודעה מוצפנת, החל מקבלת התרעה מהשרת, דרך שליפת ההודעה המוצפנת מבסיס הנתונים ופענוחה באמצעות מפתח AES-256. התהליך כולל אימות תקינות ההודעה ועדכון סטטוס "נקראה" במערכת.



ניהול קבוצות (Group Management)

תרשים מורכב המציג את יצירת קבוצה חדשה, כולל יצירת מפתח קבוצתי ייחודי ושליחת הזמנות לחברים פוטנציאליים. התהליך מבטיח שכל חבר מקבל את המפתח הקבוצתי המוצפן רק לאחר אישור הצטרפותו, ומעדכן את מסד הנתונים בהתאם.



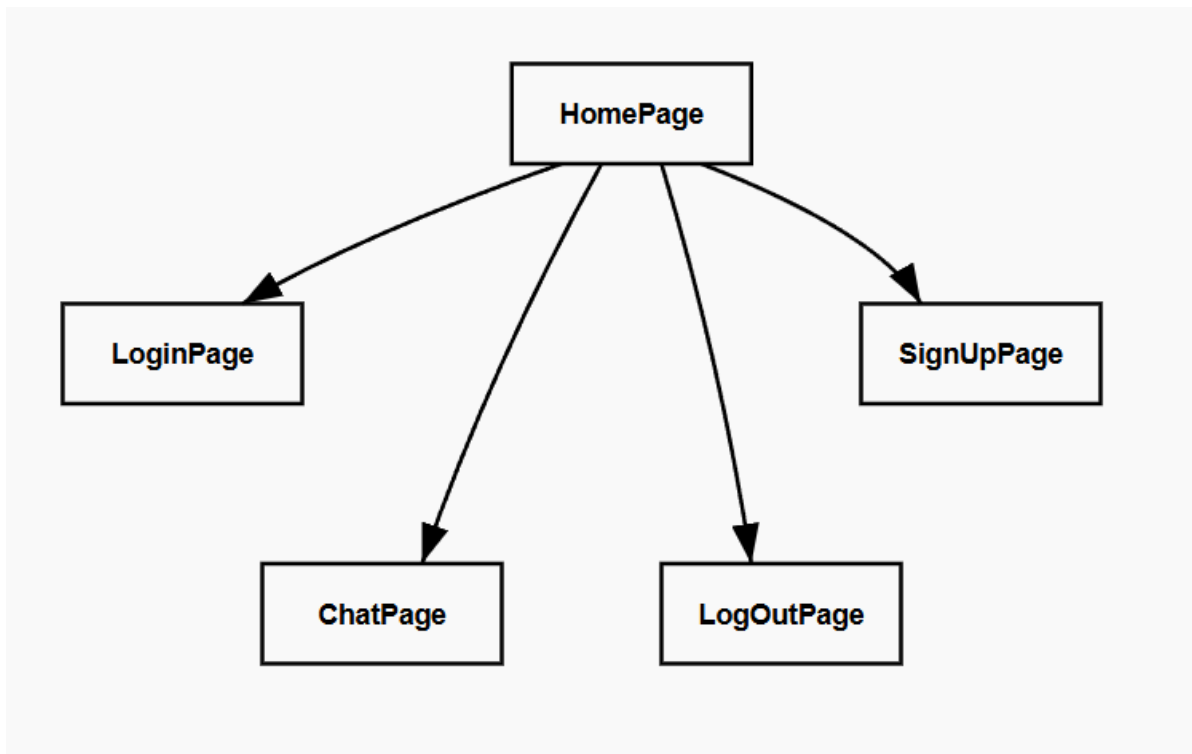
תרשים המתאר את תהליך אחזור והצגת היסטוריית השיחות, כולל שליפת הודעות מוצפנות מבסיס הנתונים, פענוח עם המפתחות המתאימים, ועיצוב התצוגה הכרונולוגית. התהליך כולל גם יכולות חיפוש וסינון מקומי בהודעות.

13. תיאור מסכים וממשק משתמש

בפרק זה נציג את מבנה המסכים במערכת SmallTalk. נתחיל בתרשים היררכיית המסכים, המציג את הקשרים בין הדפים השונים, ולאחר מכן נסביר בקצרה את תפקודו של כל מסך במערכת. פרק זה חשוב להבנת הזרימה והמעבר בין המסכים מהיבט חוויית המשתמש.

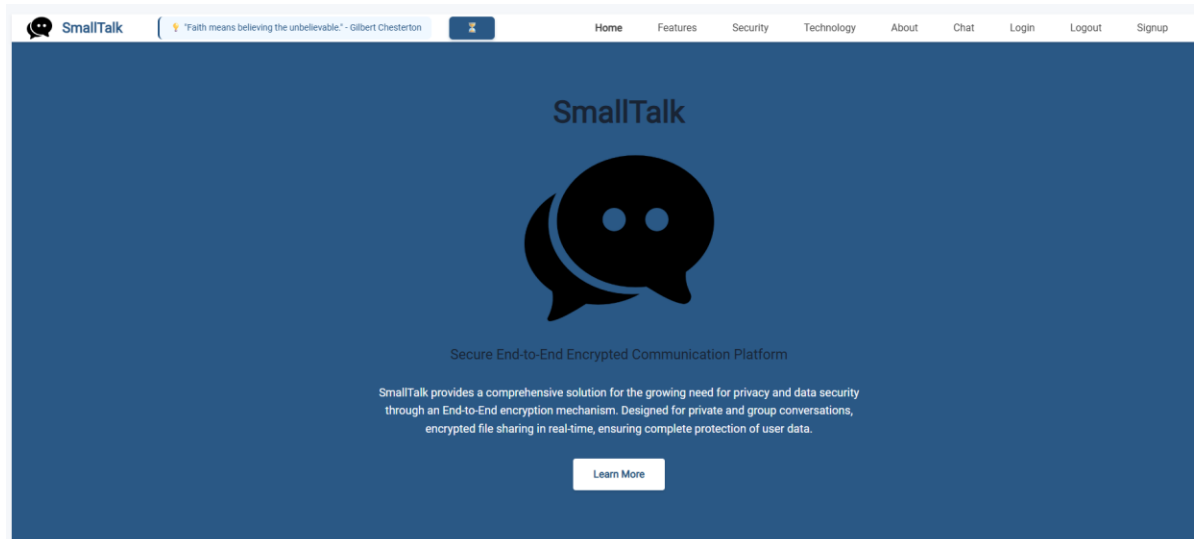
13.1 תרשים היררכיית המסכים

להלן תרשים היררכיית מסכים (Screen Flow Diagram)



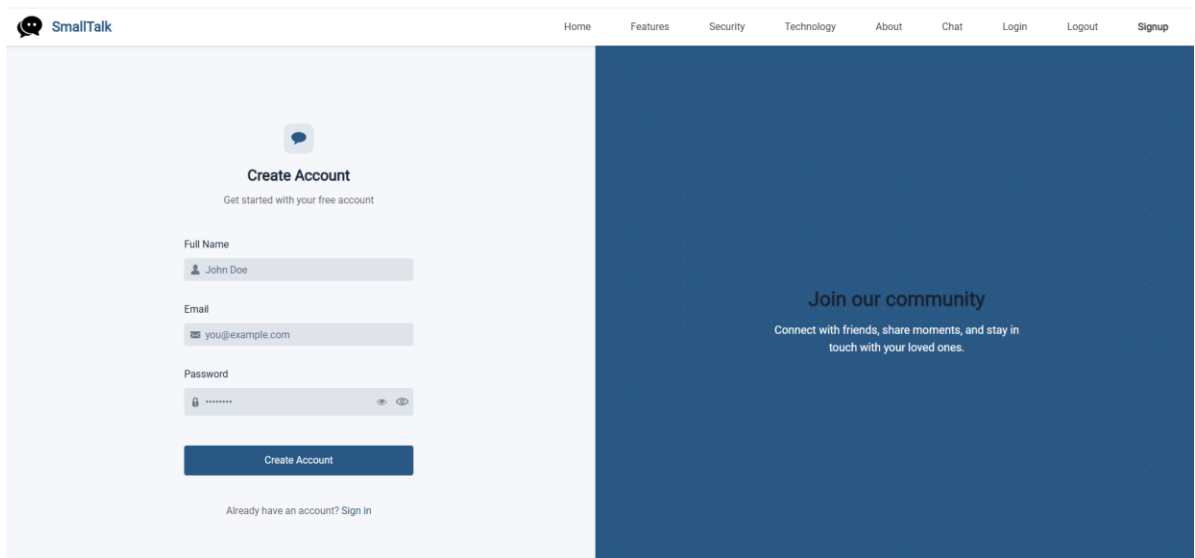
האיור מציג מבנה ירושה בתכנות מונחה עצמים, כאשר HomePage משמשת כמחלקת אב שממנה כל הדפים האחרים יורשים. החצים מראים שכל דף (LoginPage, SignUpPage, ChatPage, LogOutPage) מקבל את התכונות והפונקציות הבסיסיות של HomePage מבנה זה מאפשר קוד חוזר ועקביות בין הדפים השונים.

13.2 תיאור המסכים



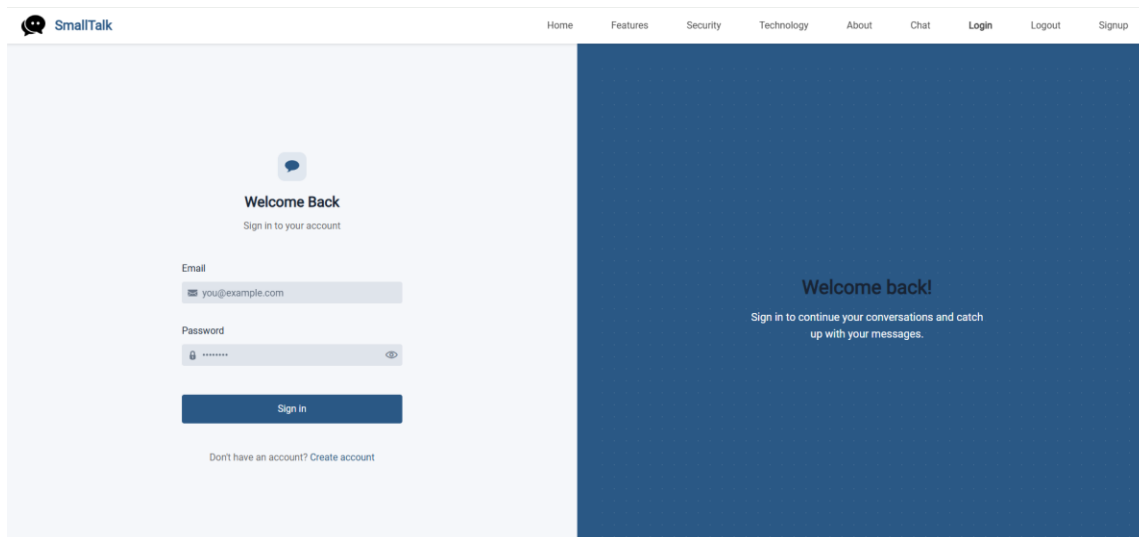
1. דף הבית (HomePage)

- מכיל תפריט ניווט עליון עם קישורים, Home, Features, Security, Technology, About, Chat, Login, Logout, Signup.
- כפתור "Learn More" שמוביל לעמוד מידע נוסף



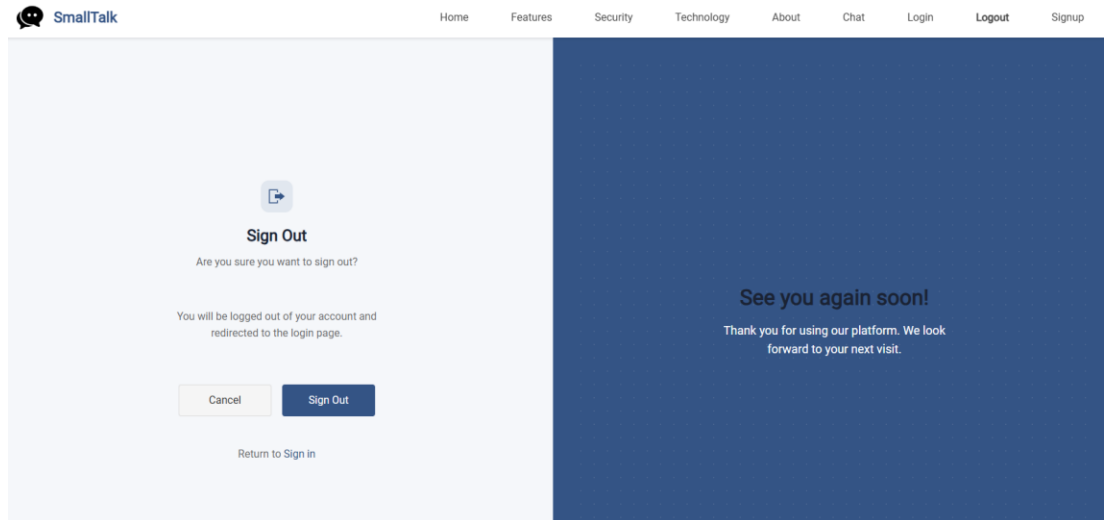
2. דף רישום (Signup)

- מכיל תפריט ניווט עליון עם קישורים, Home, Features, Security, Technology, About, Chat, Login, Logout, Signup.
- מכיל כפתור שליצור חשבון
- דף Signup אשר אתה יכול דרכו להירשם לאתר – ולכתוב אימייל שם וסיסמה למשתמש



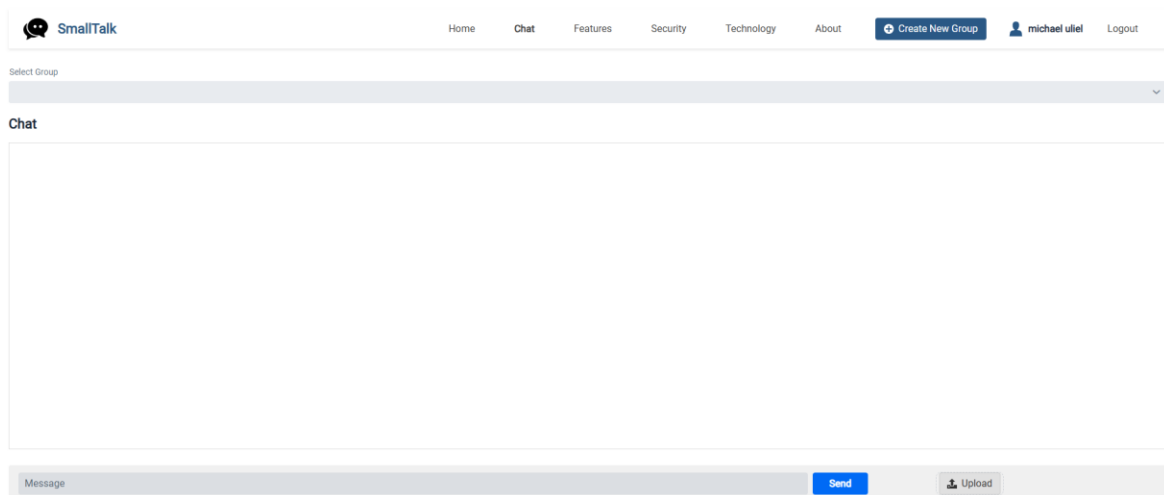
3. דף התחברות (LoginPage)

- מכיל תפריט ניווט עליון עם קישורים, Home, Features, Security, Technology, About, Chat, Login, Logout, Signup.
- מכיל כפתור בשם sign in שדרכו תוכל להתחבר למערכת
- דף LOGIN אשר דרכו יכול לבצע התחברות לאתר – חייב לרשום אימייל וסיסמה ואם אין לך משתמש תוכל ללחוץ על Create account וזה יעביר אותך לדף רישום



4. דף התנתקות (LogoutPage)

- מכיל תפריט ניווט עליון עם קישורים, Home, Features, Security, Technology, About, Chat, Login, Logout, Signup.
- מכיל כפתור בשם sign out שדרכו תוכל להתנתק מן המערכת
- דף Logout אשר אתה יכול להתנתק מהמערכת



5. דף צאט (ChatPage)

- מכיל תפריט ניווט עליון עם קישורים, Home, Features, Security, Technology, Chat, Login, Logout, Signup. של המשתמש
- מכיל כפתור בשם create new group שדרכו תוכל ליצור קבוצה
- מכיל כפתור Upload שדרכו תוכל להעלות תמונות וקבצי אודיו
- מכיל כפתור בשם Send שתוכל באמצעותו לשלוח את ההודעה
- דף chat אשר אתה יכול לדבר ולשוחח ולשלוח לתמונות לקבוצות שאתה שייך אליה

14. תיאור התוכנה

בפרק זה נציג את תיאור התוכנה ומרכיביה, כולל סביבת הפיתוח, שפות התכנות בהן נעשה שימוש, מבנה המערכת ומודוליה.

נבחן את מבני הנתונים המרכזיים ואת האופן בו הם תומכים בפעולת המערכת, תוך פירוט של מחלקות עיקריות.

לבסוף, נציג קטעי קוד נבחרים המיישמים פעולות קריפטוגרפיות ואלגוריתמים חשובים בפרויקט.

14.1 סביבת עבודה

סביבת העבודה בה השתמשתי הוא IDE – Code Visual Studio
 Windos 11 pro
 השתמשתי ב Spring boot – גרסה 3.4.4
 השתמשתי ב Vaadin גרסה - 24.6.4
 אחסון בענן - MongoDB

14.2 שפות תכנות

השתמשתי לפרוייקט בשפת java גרסה 17
 ולהלן הסבר על שפת java
 Java היא שפת תכנות עילית, מונחית עצמים, עצמאית מפלטפורמה, עם ניהול זיכרון אוטומטי ואבטחה מובנית.

היא מתאימה לפיתוח מערכות ארגוניות, אינטרנט ומובייל, אך פחות אידאלית ליישומים שדורשים ביצועים גבוהים או שליטה בחומרה.

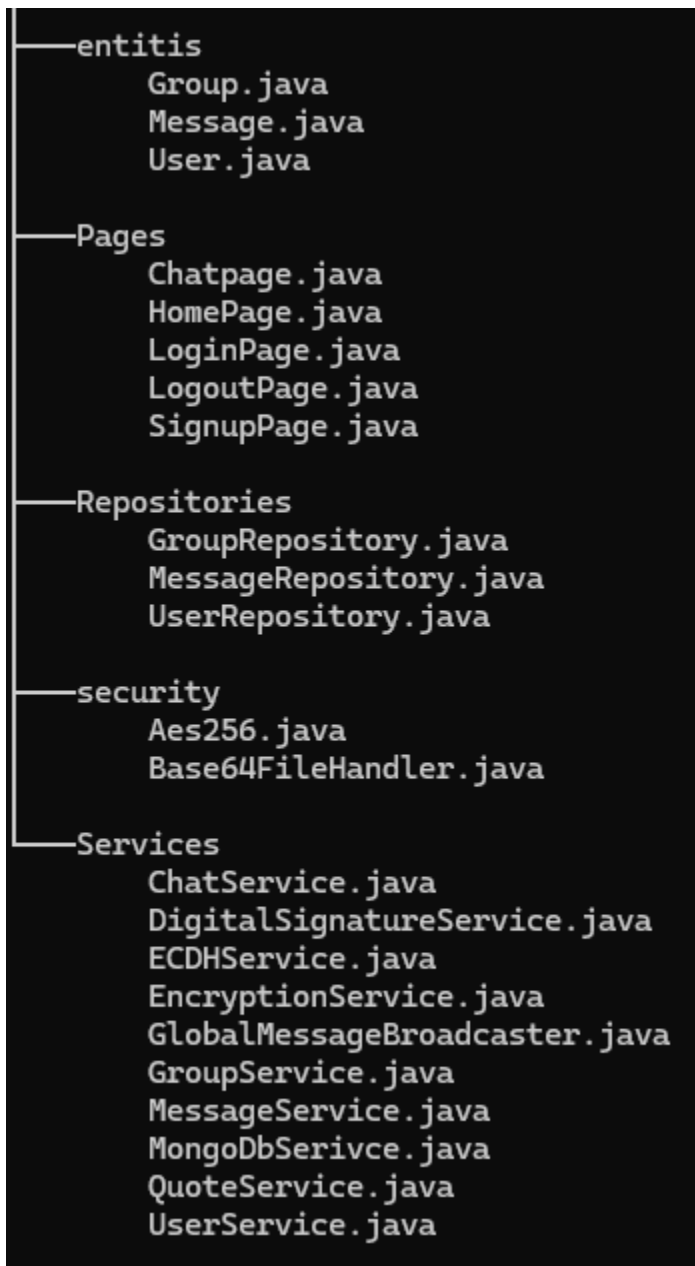
ל Java-יתרונות כמו קהילה פעילה, תאימות לאחור וספריות רבות, אך גם חסרונות כמו ביצועים איטיים יחסית ותחביר מסורב

14.3 תיאור המודולים והמחלקות

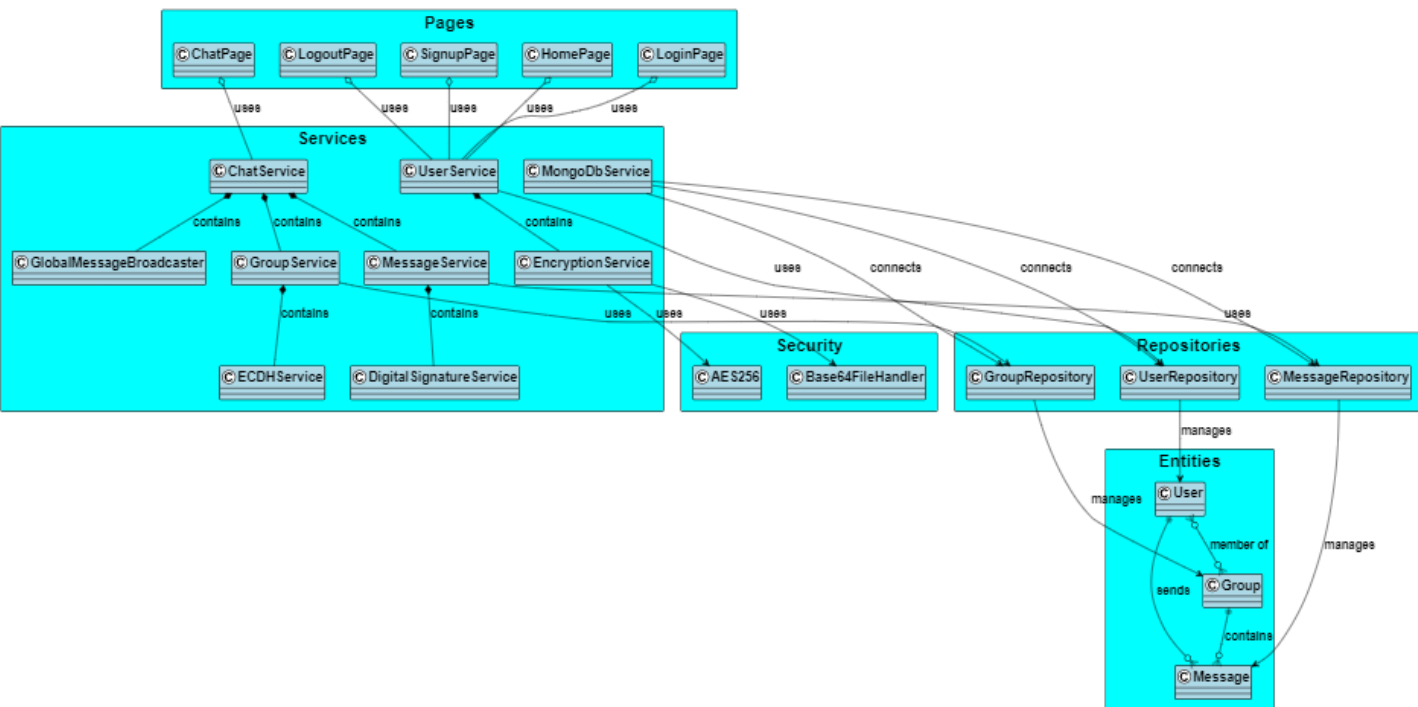
14.3.1 עץ המודולים

התמונה מציגה את מבנה הפרויקט של מערכת Java לפי תיקיות וקבצי קוד. כל תיקייה מייצגת שכבה שונה במערכת entities: למודלי נתונים Pages, לממשק משתמש, Repositories לגישה לבסיס נתונים security, להצפנה ואבטחה, ו-Services ללוגיקת שירותים.

המבנה הזה תומך בארכיטקטורה מודולרית ונוחה לתחזוקה.



14.3.2 תרשים מחלקות Class Diagram



14.3.3 תיאור מחלקות מפורט

הסבר :

HomePage - מחלקה האחראית על

יצירת דף הבית של האפליקציה בונה מבנה דף עם סקציות שונות כמו, hero תכונות, אבטחה וטכנולוגיה מספקת ממשק ניווט ותצוגה ראשונית למשתמשים חדשים

C HomePage
● HomePage() ■ createNavigationBar() : HorizontalLayout ■ createHeroSection() : Div ■ createFeaturesSection() : Div ■ createSecuritySection() : Div ■ createTechnologySection() : Div ■ createFooter() : Div ■ createFeatureBox(String title, String content) : Div

C Chatpage
○ Chatpage(ChatService chatService) ○ beforeEnter(BeforeEnterEvent event) ○ handleChatUpdate(String timestamp) ○ checkForNewMessages() ○ ensureRegistration() ◇ onAttach(AttachEvent attachEvent) ◇ onDetach(DetachEvent detachEvent) □ createNavigationBar() □ createMessageInputWithUpload() □ initializeChatForAuthenticatedUser() □ loadUserGroups(User currentUser) □ setupUserInterface(User currentUser, List<Group> userGroups) □ setupUserStatusUpdateTask() □ addMessageToDatabase(String formattedMessage, String groupId) □ setupMessageHandler() □ setupCrossBrowserCommunication() □ sendGroupMessage(String message, String groupId) □ addMessageToChat(String message) □ scrollToBottom() □ loadExistingMessages() □ showLoadingIndicators(boolean show) □ refreshChatHistory() □ decryptAndDisplayMessage(String encodedMessage, String groupId) □ initializeEncryption() □ initializeSessionId() □ updateSessionIdDisplay(String id) □ updateUserList() □ configureMediaUpload(Upload upload, MemoryBuffer buffer) □ createNewGroup() □ updateGroupSelector() □ registerWithBroadcaster()

הסבר:

Chatpage מחלקה המטפלת בממשק הצ'אט הראשי של האפליקציה כוללת פונקציות לניהול

הודעות, הצפנה, העלאת קבצים וניהול קבוצות מספקת את כל הפונקציונליות הבסיסית לחווית

הצ'אט של המשתמש

C LoginPage
userService : UserService emailField : EmailField passwordField : PasswordField
LoginPage(UserService userService) createNavigationBar() : HorizontalLayout createLoginFormColumn() : VerticalLayout validateForm() : boolean createImageColumn() : VerticalLayout

הסבר :

LoginPage מחלקה לניהול דף ההתחברות למערכת כוללת שדות עבור אימייל וסיסמה עם תהליך אימות מספקת ממשק התחברות עם עמודות נפרדות לטופס ולתמונה

C LogoutPage
userService : UserService
LogoutPage(UserService userService) createNavigationBar() : HorizontalLayout createLogoutColumn() : VerticalLayout createImageColumn() : VerticalLayout

הסבר :

LogoutPage מחלקה פשוטה לטיפול בתהליך ההתנתקות מהמערכת יוצרת דף התנתקות עם עמודות לתוכן ולתמונה מנהלת את הליך היציאה הבטוחה של המשתמש מהאפליקציה

C SignupPage
userService : UserService
SignupPage(UserService userService) createNavigationBar() : HorizontalLayout createSignupFormColumn() : VerticalLayout createImageColumn() : VerticalLayout

הסבר :

SignupPage מחלקה לניהול דף הרשמה למשתמשים חדשים יוצרת טופס הרשמה עם עמודות נפרדות לטופס ולתמונה מספקת את הממשק להקמת חשבונות משתמש חדשים במערכת

C Group
- name : String - users : ArrayList<String> - memberIds : ArrayList
- Group() - Group(String name) - addUserId(String userId) : void - getName() : String - getId() : String - setName(String name) : void - getUsers() : ArrayList<String> - setUsers(ArrayList<String> users) : void - getMemberIds() : List<String> - toString() : String

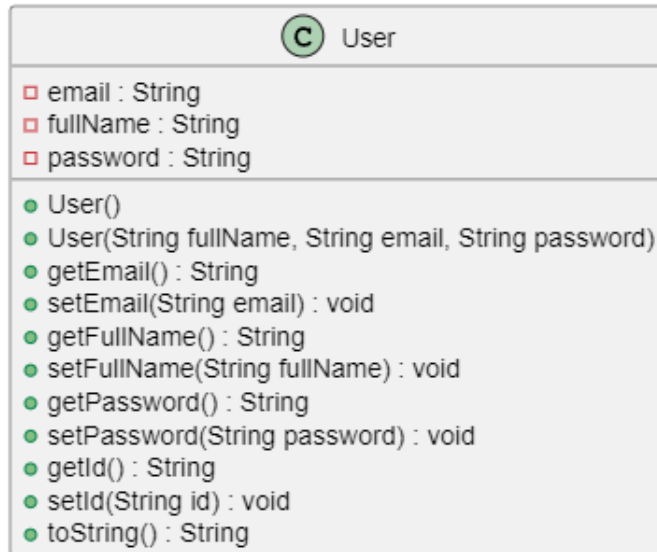
הסבר :

Group מחלקה המייצגת קבוצת צ'אט במערכת מכילה מידע על שם הקבוצה, רשימת משתמשים ומזהים מספקת מתודות לניהול חברי הקבוצה והגדרותיה

C Message
- senderID : String - receiverID : String - textContent : byte[] - mediaContent : byte[] - mediaContentType : String - chatId : String - time : LocalDateTime
- Message() - Message(String sender, String receiver, byte[] textContent, byte[] mediaContent, String mediaContentType, String chatId) - getSenderID() : String - getReceiverID() : String - getChatId() : String - getTimestamp() : LocalDateTime - setSenderID(String sender) : void - setReceiverID(String receiver) : void - setChatId(String chatId) : void - getTime() : LocalDateTime - setTime(LocalDateTime time) : void - getTextContent() : byte[] - setTextContent(byte[] textContent) : void - getMediaContent() : byte[] - setMediaContent(byte[] mediaContent) : void - getMediaContentType() : String - setMediaContentType(String mediaContentType) : void

הסבר :

Message מחלקה המייצגת הודעה במערכת הצ'אט מכילה מידע על השולח, הנמען, תוכן טקסטואלי ומדיה מנהלת זמני שליחה, הצפנה ופרטי ההודעה השונים



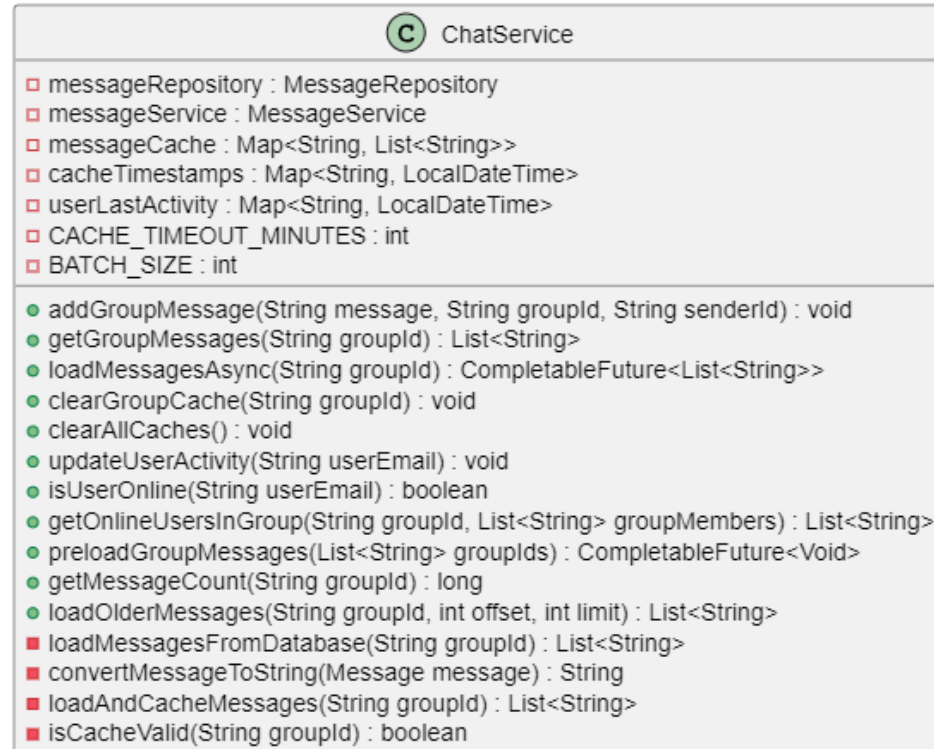
הסבר :

User מחלקה בסיסית המייצגת משתמש במערכת מכילה מידע אישי כמו שם מלא, אימייל וסיסמה מספקת מתודות לגישה ועדכון של פרטי המשתמש



הסבר :

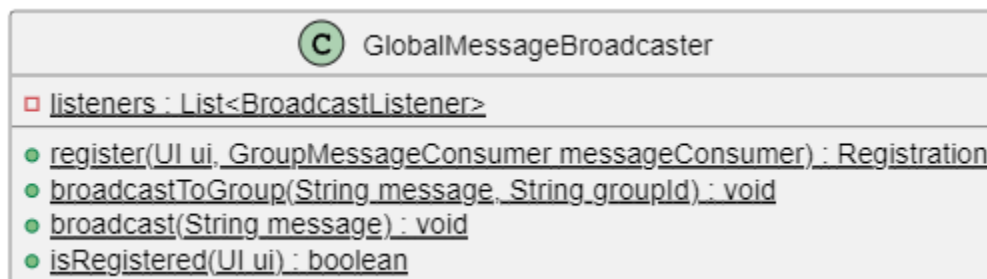
Aes256 מחלקה המממשת הצפנה בסטנדרט-AES כוללת כל הפונקציות הדרושות להצפנה ופענוח של מידע מנהלת את מפתחות ההצפנה ותהליכי האבטחה הקריפטוגרפיים

**הסבר :**

ChatService שירות מרכזי לניהול פעילות הצ'אט במערכת מטפל בשמירה, טעינה ומטמון של הודעות קבוצתיות מנהל פעילות משתמשים ותקשורת בזמן אמת

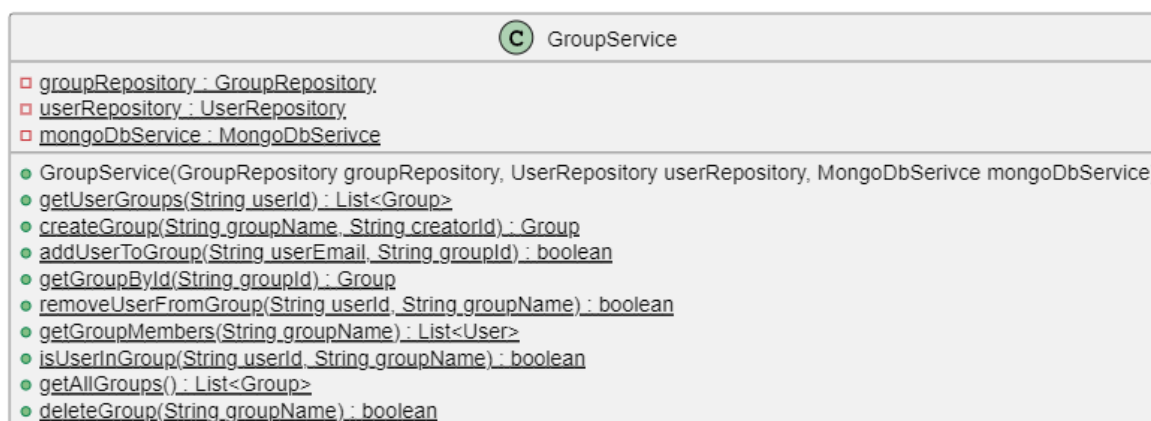
**הסבר :**

EncryptionService שירות המספק יכולות הצפנה מתקדמות למערכת משתמש במנוע-AES 256 לעיבוד אסינכרוני של מידע מוצפן מנהל את תהליכי ההצפנה והפענוח של הודעות והקבצים



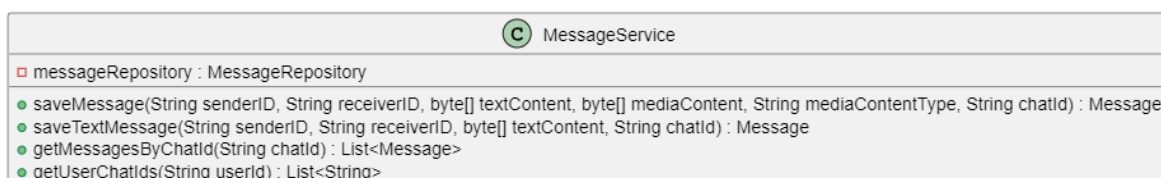
הסבר :

GlobalMessageBroadcaster שירות לשידור הודעות גלובלי לכלל המשתמשים המחוברים מנהל רשימת מאזינים ושולח הודעות לקבוצות ולכלל המערכת מספק תשתית תקשורת המותאמת לאפליקציות צ'אט בזמן אמת



הסבר :

GroupService שירות לניהול קבוצות הצ'אט במערכת מטפל ביצירה, עדכון ומחיקה של קבוצות ומשתמשים מתממשק עם מסד הנתונים לשמירת מידע הקבוצות



הסבר :

MessageService שירות לניהול הודעות במערכת הצ'אט שומר ומשליף הודעות ממסד הנתונים לפי צ'אט או משתמש מספק ממשק לעבודה עם הודעות טקסטואליות במערכת

C MongoDBService	
□	userRepo : UserRepository
□	groupRepo : GroupRepository
●	MongoDbService(UserRepository userRepo, GroupRepository groupRepo)
●	addUserToDB(User user) : void
●	addGroupToDB(Group group) : void
●	getUserGroups(String userId) : List<Group>

הסבר :

MongoDbService שירות למתן ממשק למסד נתונים MongoDB מטפל בפעולות בסיסיות של הוספה ושליפה של משתמשים וקבוצות מספק שכבת גישה למסד הנתונים עבור יתר השירותים

C UserService	
□	<u>userRepository</u> : UserRepository
□	<u>passwordEncoder</u> : BCryptPasswordEncoder
○	<u>USER_KEY</u> : String
○	<u>USERNAME_KEY</u> : String
●	UserService(UserRepository userRepository)
●	<u>isUserAuthenticated()</u> : boolean
●	<u>getAuthenticatedUser()</u> : User
●	<u>clearAuthenticatedUser()</u> : void
●	registerUser(String fullName, String email, String password) : User
●	authenticateUser(String email, String password) : User
●	getCurrentUser() : User
●	logout() : void
●	updateProfilePicture(User user) : void
●	updateProfilePicture(String userId, String profilePicData) : void
●	updateProfilePictureByEmail(String email, String profilePicData) : void
●	<u>getUserById(String email)</u> : User
●	<u>getUserByEmail(String email)</u> : User
●	deleteUserByEmail(String email) : void
●	<u>getAllUsers()</u> : List<User>

הסבר :

UserService שירות מרכזי לניהול משתמשים במערכת מטפל באימות, הרשמה, התחברות ועדכון פרופילים מנהל הרשאות משתמשים ואת מצב ההתחברות שלהם למערכת

14.4 מבנה נתונים בשימוש

פרויקט נעשה שימוש במבני נתונים שונים, בהתאם לצורך של כל רכיב במערכת. להלן הסבר על מבני הנתונים המרכזיים:

1. ישויות (Entities)

• Group.java, Message.java, User.java

- אלו מחלקות המייצגות את הישויות המרכזיות במערכת (קבוצות, הודעות, משתמשים). כל מחלקה מכילה שדות (fields) המתארים את המאפיינים של הישות, לדוג' ב User.java - יהיו שדות כמו password, username, userId וכו'.
- בדרך כלל, מבני הנתונים כאן הם אובייקטים (Objects) של המחלקות הנ"ל, ולעיתים נעשה שימוש ברשימות (List), מערכים (Array), או מפות (Map) לאחסון אוספים של ישויות.

2. עמודים (Pages)

- המחלקות בעמוד זה) כגון ChatPage, HomePage וכו' (משתמשות לרוב במבני נתונים לאחסון זמני של נתונים הקשורים לממשק המשתמש, כמו רשימות הודעות (List<Message>), אובייקטי משתמש נוכחי (User), או מפות (Map) לאחסון נתונים דינמיים.

3. מאגרים (Repositories)

- מחלקות אלה אחראיות על ניהול הנתונים מול מסד הנתונים (Database). לרוב, הן משתמשות באוספים כמו List, Set או Map כדי לאחסן תוצאות של שאילתות, לדוג' Map<String, Group> או List<User>.

4. שירותי אבטחה (Security)

- מחלקות כמו Aes256 או Base64FileHandler עושות שימוש במבני נתונים כמו מערכים של בתים (byte[]) לצורך הצפנה/פענוח של נתונים, וכן מחרוזות (String) לאחסון מפתחות או נתונים מקודדים.

5. שירותים (Services)

- שירותים שונים משתמשים במבני נתונים מגוונים:
- **רשימות (List):** לאחסון הודעות, משתמשים, קבוצות וכו'.
- **מפות (Map):** לשיוך בין מזהים לאובייקטים (למשל. Map<String, User>).
- **מערכים (Array):** לעיבוד נתונים מוצפנים או קבצים.
- **Set :** לשמירה על ערכים ייחודיים (למשל, מזהי משתמשים בקבוצה).

14.5 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

פעולה ראשונה - הצפנת בלוק אחד (**encrypt**)

```
public byte[] encrypt(byte[] input){
    if (input.length != 16){
        throw new IllegalArgumentException("Input length must be 16 bytes");
    }
    byte[] output = new byte[16]
    int[][] state = new int[4][4]

    for (int i = 0; i < 4; i++){
        for (int j = 0; j < 4; j++){
            state[j][i] = input[i * 4 + j] & 0xff;
        }
    }

    addRoundKey(state, expandedKey, 0);
    for (int round = 1; round < Nr; round++){
        subBytes(state);
        shiftRows(state);
        mixColumns(state);
        addRoundKey(state, expandedKey, round);
    }

    subBytes(state);
    shiftRows(state);
    addRoundKey(state, expandedKey, Nr);
    for (int i = 0; i < 4; i++){
        for (int j = 0; j < 4; j++){
            output[i * 4 + j] = (byte) state[j][i];
        }
    }

    return output;
}
```

טענת כניסה:

- פרמטר – `byte[] input`: בלוק בגודל 16 בתים להצפנה.

טענת יציאה:

- ערך מוחזר – `byte[]`: בלוק מוצפן בגודל 16 בתים.

מה מבצע?

- מצפין בלוק אחד של נתונים בגודל 16 בתים לפי תקן AES-256.

אופן פעולה:

1. ממפה את המערך למטריצת מצב. (`state`)
2. מוסיף מפתח ראשוני. (`addRoundKey`)
3. מבצע 14 סבבים של פעולות. (`subBytes`, `shiftRows`, `mixColumns`, `addRoundKey`)
4. מחזיר את הבלוק המוצפן.

סיבוכיות זמן ריצה:

- $O(1)$ זמן ריצה קבוע לכל בלוק (16 בתים).

פעולה שנייה - הרחבת מפתח (`expandKey`)

```
private int[] expandKey(byte[] key) {
    int[] w = new int[Nb * (Nr + 1)];
    int temp;
    int i = 0;
    while (i < Nk) {
        w[i] = ((key[4 * i] & 0xff) << 24) |
            ((key[4 * i + 1] & 0xff) << 16) |
            ((key[4 * i + 2] & 0xff) << 8) |
            (key[4 * i + 3] & 0xff);
        i++;
    }
    i = Nk;
    while (i < Nb * (Nr + 1)) {
        temp = w[i - 1];
        if (i % Nk == 0) {
            temp = subWord(rotWord(temp)) ^ RCON[i / Nk - 1];
        } else if (Nk > 6 && i % Nk == 4) {
```

```

        temp = subWord(temp);
    }
    w[i] = w[i - Nk] ^ temp;
    i++;
}
return w;
}

```

טענת כניסה:

- פרמטר – `byte[] key`: מפתח בגודל 32 בתים.

טענת יציאה:

- ערך מוחזר – `int[]`: מפתח מורחב (expanded key) לכל הסבבים.

מה מבצע?

- יוצר מפתחות עגולים (round keys) לכל סבבי ההצפנה.

אופן פעולה:

1. ממיר את המפתח הראשוני למערך מילים. (words)
2. מרחיב את המפתח ע"י חישוב מילים נוספות לפי אלגוריתם AES.
3. מחזיר את המפתח המורחב.

סיבוכיות זמן ריצה:

- $O(1)$ מספר פעולות קבוע (תלוי בגודל המפתח).

פעולה שלישית :

הצפנת מחרוזת (encryptString)

```

public byte[] encryptString(String text){
    byte[] textBytes = text.getBytes(StandardCharsets.UTF_8);
    byte[] paddedInput = padInput(textBytes);
    byte[] result = new byte[paddedInput.length];

    for (int i = 0; i < paddedInput.length; i += 16){
        byte[] block = Arrays.copyOfRange(paddedInput, i, i + 16);
        byte[] encryptedBlock = encrypt(block);
        System.arraycopy(encryptedBlock, 0, result, i, 16);
    }
}

```

```

return result;
{

```

טענת כניסה:

- פרמטר – String text: מחרוזת להצפנה.

טענת יציאה:

- ערך מוחזר – byte[]: מחרוזת מוצפנת כמערך בתים.

מה מבצע?

- מצפין מחרוזת (בכל אורך) ע"י חלוקה לבלוקים של 16 בתים והצפנה של כל בלוק.

אופן פעולה:

1. ממיר את המחרוזת לבתים.
2. מוסיף ריפוד (padding) לבלוק שלם.
3. מצפין כל בלוק בנפרד.
4. מחזיר את כל המידע המוצפן.

סיבוכיות זמן ריצה:

- $O(n)$ כאשר n הוא אורך המחרוזת.

פעולה רביעית :

פענוח מחרוזת (decryptToString)

```

public String decryptToString(byte[] encrypted) {
    if (encrypted.length % 16 != 0) {
        throw new IllegalArgumentException("Encrypted data length must be multiple
of 16");
    }
    byte[] decrypted = new byte[encrypted.length];
    for (int i = 0; i < encrypted.length; i += 16) {
        byte[] block = Arrays.copyOfRange(encrypted, i, i + 16);
        byte[] decryptedBlock = decrypt(block);
        System.arraycopy(decryptedBlock, 0, decrypted, i, 16);
    }
    return new String(removePadding(decrypted), StandardCharsets.UTF_8);
}

```

טענת כניסה:

- פרמטר – `byte[] encrypted` מחרוזת מוצפנת כמערך בתיים.

טענת יציאה:

- ערך מוחזר – `String` מחרוזת מקורית לאחר פענוח.

מה מבצע?

- מפענח מחרוזת מוצפנת (בכל אורך), מסיר ריפוד ומחזיר את הטקסט המקורי.

אופן פעולה:

1. בודק שהאורך תקין.
2. מפענח כל בלוק בנפרד.
3. מסיר ריפוד.
4. ממיר למחרוזת.

סיבוכיות זמן ריצה:

- $O(n)$ כאשר n הוא אורך המחרוזת המוצפנת.

פעולה חמישית:

הצפנת קובץ (`encryptFile`)

```
public void encryptFile(File inputFile, File outputFile) throws IOException {
    byte[] fileContent = Files.readAllBytes(inputFile.toPath());
    int paddedLength = ((fileContent.length + 15) / 16) * 16;
    byte[] paddedContent = new byte[paddedLength];
    System.arraycopy(fileContent, 0, paddedContent, 0, fileContent.length);

    byte[] encryptedContent = new byte[paddedLength];
    for (int i = 0; i < paddedLength; i += 16) {
        byte[] block = new byte[16];
        System.arraycopy(paddedContent, i, block, 0, 16);
        byte[] encryptedBlock = encrypt(block);
        System.arraycopy(encryptedBlock, 0, encryptedContent, i, 16);
    }
    Files.write(outputFile.toPath(), encryptedContent);
}
```


טענת כניסה:

- פרמטרים:
- – File inputFile קובץ מקור להצפנה
- – File outputFile קובץ יעד לכתיבת התוצאה המוצפנת

טענת יציאה:

- אין ערך מוחזר (void) הקובץ המוצפן נכתב לדיסק.

מה מבצע?

- מצפין קובץ שלם בבלוקים של 16 בתיים.

אופן פעולה:

1. קורא את תוכן הקובץ.
2. מוסיף ריפוד.
3. מצפין כל בלוק בנפרד.
4. כותב את התוצאה לקובץ חדש.

סיבוכיות זמן ריצה:

- $O(n)$ כאשר n הוא גודל הקובץ.

15. מדריך למשתמש

על מנת להריץ את המערכת תצטרך קודם כל שהפריויקט יהיה אצלך במחשב עליך להוריד למחשב – vscode ואז תצטרך להריץ את השרת. ואז לרשום בדפדפן שיש לך localhost:8080 שתיכנס לדף הבית יהיה לך שתי אפשרויות – הרשמה לאתר או כניסה אם יש משתמש קיים לאחר שנכנסת למערכת ויש לך משתמש יהיה לך מספר אפשרויות. יש Navbar שמשמש כניווט ותפריט בכל הדפים האלה אפשר לחזור ולהגיע לכל דף דף בית – עם הסבר על האתר דף להתנתק – תוכל להתנתק מהמשתמש שלך דף להתחבר – תוכל להתחבר עם יש לך אימייל וסיסמא אם לא תיצור חשבון דף להירשם – דף שבו תוכל להירשם למערכת דף צאט – שם תוכל לדבר ולשוחח ולראות את הקבוצות

16. בדיקות והערכה

למען בדיקת אמינות המערכת בוצעו בפרויקט זה מספר רחב של בדיקות שנועדו לוודא שהמערכת פועלת כהלכה ולא בוצעו בה שגיאות לא רצויות בצד הלקוח ובצד השרת. להלן חלק מהבדיקות אותם ביצעתי בפרויקט התחברות למערכת ובדיקת כל ההרשאות של הפרויקט. עברתי על כל הבקשות המתבצעות בצד הלקוח הדורשים הרשאות משתמש ובדקתי האם השרת פועל לפי ההרשאות של כל משתמש וחוסם דברים שלמשתמש לא צריך להיות גישה אליהם. העברת בקשות התחברות שונות הכוללות בקשות HTTP והתחברות מכמה משתמשים Tomcat (בעל השרת) בדיקת שליחת בקשות לשרת ממספר משתמשים בו זמנית לבדיקה האם השרת יכול לטפל בכל הבקשות בו זמני.

17. מסקנות

במהלך העבודה על הפרויקט, התמודדתי לראשונה עם אתגרי ניהול זמן משמעותיים. נדרשתי לעמוד בלוחות זמנים צפופים, לשלב בין דרישות הפרויקט לבין משימות שוטפות בלימודים, ולעבוד במקביל על כתיבת הדוח, הפיתוח והמחקר. ההתמודדות הזו חידדה אצלי את החשיבות של תכנון נכון, חלוקת משימות, ועבודה מסודרת לפי סדרי עדיפויות.

במהלך הפרויקט נחשפתי לטכנולוגיות חדשות, במיוחד לסביבת Spring Boot ולעבודה מול MongoDB. למדתי כיצד לממש פעולות CRUD (יצירה, קריאה, עדכון ומחיקה) מול השרת, ולהבין לעומק כיצד כל התהליך הזה פועל מאחורי הקלעים. תהליך הלמידה כלל חקירה עצמאית, קריאה ממקורות שונים, והשוואה בין גישות טכנולוגיות כדי לבחור את הפתרון המתאים ביותר לפרויקט ולצרכים שלי.

בנוסף, למדתי להיעזר בחברי לכיתה בעת הצורך, וגם לתרום מהידע שלי לאחרים כאשר התאפשר לי. הבנתי שלמידה משותפת ושיתוף פעולה בצוות לא רק מקלים על ההתמודדות עם תקלות ובעיות, אלא גם מעשירים את הידע והניסיון של כל אחד מאיתנו.

העמקתי בחשיבות המחקר העצמאי – גיליתי שכלל שמעמיקים בנושא מסוים, כך אפשר להגיע להבנה רחבה יותר, ולשלב בין מקורות מידע שונים כדי ליצור פתרון איכותי ומותאם אישית. למדתי להציג ולהסביר את הפתרונות שמצאתי, לפרט את שלבי העבודה, ולהנגיש את המערכת שבניתי כך שתהיה ברורה ומובנת למשתמשים.

לבסוף, ההתמודדות עם תקלות טכניות, הן שלי והן של אחרים, חיזקה אצלי את החשיבות של עבודת צוות, סבלנות, והיכולת לחלוק ידע וניסיון – תכונות חיוניות להצלחה בכל פרויקט טכנולוגי.

18. פיתוחים עתידיים

1. הפיצ'ר סטטוס אישי דינמי (כמו בוואטסאפ):

- יאפשר למשתמשים להעלות עדכון אישי זמני (טקסט/תמונה/סרטון)
- העדכון יוצג לחברים/קשרים במערכת למשך 24 שעות
- נותן הצצה מהירה למה שקורה לאנשים בזמן אמת.

2. שיחה קולית/וידאו הטמעת אפשרות לתקשורת סינכרונית דרך שיחות קוליות או וידאו תשדרג משמעותית את יכולות התקשורת במערכת, ותאפשר אינטראקציה אנושית יותר וישירה

3. יכולות תרגום מקוונות בזמן אמת הוספת מנגנון תרגום אוטומטי מידי שיאפשר תקשורת בין שפות ללא מגבלות. המערכת תתרגם הודעות באופן אוטומטי בזמן אמת, מה שיפרוץ מחסומי שפה ויאפשר תקשורת גלובלית יותר.

19. בבליוגרפיה

1. Robby K. H, Antonius I.S, Dyah N, Widyastuti W, "Securing RFID in IoT Networks with Lightweight AES and ECDH Cryptography Approach", August 2024
<https://www.researchgate.net/publication/383236735>

2. Limin Zhang, Li Wang, "A Hybrid Encryption Approach for Efficient and Secure Data Transmission in IoT Devices", June 2024
<https://jeas.springeropen.com/articles/10.1186/s44147-024-00459-x>

3. הסבר על אלגוריתם AES 256

<https://www.wallarm.com/what/what-is-aes-advanced-encryption-standard>

4. הסבר על אלגוריתם ECDH

<https://how.dev/answers/what-is-the-elliptic-curve-diffie-hellman-algorithm>

5. שירות חיצוני (rest api)

<https://zenquotes.io/api/random>